

# EpiGraph Epidemic Intervention Planner

Comprehensive Technical, Mathematical, and Decision Review

---

Author: Varad Lingayat

Academic context: Second-year discrete mathematics project

Version: Final technical review

Date: 21 August 2026

**Scope notice.** This document describes an educational scenario model. It does not provide public-health forecasts, clinical guidance, or operational recommendations. Real-road geometry may be imported from OpenStreetMap; location populations, contact probabilities, and disease outcomes remain explicit synthetic model inputs.

# Contents

|     |                                                                             |    |
|-----|-----------------------------------------------------------------------------|----|
| 1   | Executive summary .....                                                     | 1  |
| 1.1 | Principal contributions .....                                               | 2  |
| 1.2 | Reading guide .....                                                         | 2  |
| 2   | Problem framing and requirements .....                                      | 3  |
| 2.1 | Academic problem statement .....                                            | 3  |
| 2.2 | Requirements translated into engineering decisions .....                    | 3  |
| 2.3 | Non-goals and boundary decisions .....                                      | 4  |
| 2.4 | Success criteria .....                                                      | 4  |
| 3   | Decision architecture: alternatives considered and selected rationale ..... | 4  |
| 3.1 | Decision philosophy .....                                                   | 4  |
| 3.2 | Major alternatives .....                                                    | 5  |
| 3.3 | Why the solution is modular .....                                           | 6  |
| 3.4 | Rejected complexity .....                                                   | 6  |
| 4   | System architecture and information flow .....                              | 6  |
| 4.1 | Logical architecture .....                                                  | 6  |
| 4.2 | Domain contract .....                                                       | 7  |
| 4.3 | Portable runtime architecture .....                                         | 7  |
| 4.4 | Trust and failure boundaries .....                                          | 7  |
| 5   | Graph construction and data model .....                                     | 8  |
| 5.1 | Formal graph representation .....                                           | 8  |
| 5.2 | Synthetic generator .....                                                   | 8  |
| 5.3 | Imported road geometry .....                                                | 9  |
| 5.4 | Actions as graph transformations .....                                      | 9  |
| 6   | Discrete epidemic mathematics .....                                         | 9  |
| 6.1 | Relationship to classical SIR .....                                         | 9  |
| 6.2 | Edge exposure and path transmission .....                                   | 10 |
| 6.3 | Binomial daily transitions .....                                            | 10 |
| 6.4 | Determinism and common random numbers .....                                 | 11 |
| 6.5 | Derived measures and interpretation .....                                   | 11 |
| 7   | Intervention algorithms and constrained optimization .....                  | 11 |
| 7.1 | Feasible action set .....                                                   | 11 |
| 7.2 | Random baseline .....                                                       | 12 |
| 7.3 | Highest degree .....                                                        | 12 |
| 7.4 | Betweenness centrality .....                                                | 12 |
| 7.5 | Dijkstra route blocking .....                                               | 12 |
| 7.6 | Max-flow/min-cut .....                                                      | 13 |
| 7.7 | Comparison objective .....                                                  | 13 |

|      |                                                                          |    |
|------|--------------------------------------------------------------------------|----|
| 8    | Bayesian hotspot scoring and actionable recommendations .....            | 14 |
| 8.1  | Why a Bayesian component was included .....                              | 14 |
| 8.2  | Prior construction .....                                                 | 14 |
| 8.3  | Evidence likelihoods and posterior .....                                 | 14 |
| 8.4  | Interpretation discipline .....                                          | 15 |
| 8.5  | Recommendations .....                                                    | 15 |
| 9    | Experimentation, fairness, sensitivity, and demonstration protocol ..... | 15 |
| 9.1  | Fair comparison protocol .....                                           | 15 |
| 9.2  | Sensitivity analysis .....                                               | 16 |
| 9.3  | Curated Demonstration Mode .....                                         | 16 |
| 9.4  | Manual what-if mode .....                                                | 17 |
| 10   | Interface, visualization, reports, and exports .....                     | 17 |
| 10.1 | Workspace design .....                                                   | 17 |
| 10.2 | 2D network explorer .....                                                | 17 |
| 10.3 | Architectural miniature 3D view .....                                    | 17 |
| 10.4 | Results, sharing, and downloads .....                                    | 18 |
| 10.5 | Theme and interaction quality .....                                      | 18 |
| 11   | Data provenance, governance, and desktop packaging .....                 | 18 |
| 11.1 | Road geometry versus model variables .....                               | 18 |
| 11.2 | Import safety measures .....                                             | 19 |
| 11.3 | Portable Windows package .....                                           | 19 |
| 11.4 | Security and privacy observations .....                                  | 20 |
| 12   | Verification, validation, and quality assurance .....                    | 20 |
| 12.1 | What was verified .....                                                  | 20 |
| 12.2 | Testable invariants .....                                                | 20 |
| 12.3 | UI interaction evidence .....                                            | 21 |
| 12.4 | Visual and runtime QA .....                                              | 21 |
| 12.5 | Quality limitations .....                                                | 21 |
| 13   | Limitations, responsible interpretation, and future work .....           | 22 |
| 13.1 | Mathematical limitations .....                                           | 22 |
| 13.2 | Statistical limitations .....                                            | 22 |
| 13.3 | Ethical limitations .....                                                | 22 |
| 13.4 | Future work, ordered by responsible value .....                          | 23 |
| 14   | Conclusion .....                                                         | 23 |
| 15   | Decision register: implementation choices and rationale .....            | 24 |
| 15.1 | A. Mathematical domain and graph foundations .....                       | 25 |
| 15.2 | B. Epidemic engine and probability choices .....                         | 26 |
| 15.3 | C. Intervention and comparison choices .....                             | 27 |

|       |                                                                                 |    |
|-------|---------------------------------------------------------------------------------|----|
| 15.4  | D. Bayesian, recommendation, and experiment choices .....                       | 28 |
| 15.5  | E. Interface, visualisation, and teaching choices .....                         | 29 |
| 15.6  | F. External-data, resilience, and desktop choices .....                         | 30 |
| 15.7  | G. Verification, release, and handoff choices .....                             | 31 |
| 16    | Appendices .....                                                                | 31 |
| 16.1  | Appendix A — Worked numerical day update .....                                  | 32 |
| 16.2  | Appendix B — Worked Bayesian update .....                                       | 33 |
| 16.3  | Appendix C — Complexity and algorithmic scale .....                             | 34 |
| 16.4  | Appendix D — Exact interface contract summary .....                             | 35 |
| 16.5  | Appendix E — Test and review matrix .....                                       | 36 |
| 16.6  | Appendix F — Bibliography and implementation evidence .....                     | 37 |
| 17    | Appendix E — Formal model, state-machine, and data-contract audit .....         | 38 |
| 17.1  | E.1 Domains, notation, and typing discipline .....                              | 38 |
| 17.2  | E.2 Parameter vector and admissible ranges .....                                | 39 |
| 17.3  | E.3 Intervention transformation as a pure graph operation .....                 | 40 |
| 17.4  | E.4 Force of infection and daily update order .....                             | 40 |
| 17.5  | E.5 Deterministic pseudo-random trials .....                                    | 41 |
| 17.6  | E.6 Derived metrics and semantic boundaries .....                               | 42 |
| 18    | Appendix F — Algorithm execution trace and strategy-by-strategy audit .....     | 43 |
| 18.1  | F.1 Shared comparison skeleton .....                                            | 43 |
| 18.2  | F.2 Random: required baseline, not an endorsement .....                         | 43 |
| 18.3  | F.3 Highest Degree: local connectivity heuristic .....                          | 44 |
| 18.4  | F.4 Betweenness: shortest-path bridge heuristic .....                           | 45 |
| 18.5  | F.5 Dijkstra route blocking: probability as additive cost .....                 | 45 |
| 18.6  | F.6 Max-flow/min-cut: structural separation heuristic .....                     | 46 |
| 18.7  | F.7 From rankings to recommendations .....                                      | 47 |
| 18.8  | F.8 Comparative interpretation matrix .....                                     | 47 |
| 19    | Appendix G — Engineering design review and module-level rationale .....         | 49 |
| 19.1  | G.1 Layered architecture and the shared-domain decision .....                   | 49 |
| 19.2  | G.2 React workspace and the five-step navigation model .....                    | 50 |
| 19.3  | G.3 State ownership, reset semantics, and side-effect control .....             | 50 |
| 19.4  | G.4 2D graph explorer: faithful interaction before visual decoration .....      | 51 |
| 19.5  | G.5 3D architectural miniature: explanatory realism, not photorealism .....     | 51 |
| 19.6  | G.6 Data import: bounded OpenStreetMap access and semantic transformation ..... | 52 |
| 19.7  | G.7 Demonstration fixtures and offline-first teaching reliability .....         | 52 |
| 19.8  | G.8 Server procedures, persistence, sharing, and explanation boundaries .....   | 53 |
| 19.9  | G.9 Export decisions: CSV, PDF, and permanent share link .....                  | 53 |
| 19.10 | G.10 Electron portable desktop package .....                                    | 53 |

|       |                                                                                         |    |
|-------|-----------------------------------------------------------------------------------------|----|
| 19.11 | G.11 Security, privacy, and non-goals .....                                             | 54 |
| 19.12 | G.12 Engineering review conclusion .....                                                | 54 |
| 20    | Appendix H — Verification catalog, quality evidence, and residual test boundaries ..... | 55 |
| 20.1  | H.1 Verification philosophy .....                                                       | 55 |
| 20.2  | H.2 Mathematical core obligations .....                                                 | 55 |
| 20.3  | H.3 Strategy-specific obligations .....                                                 | 56 |
| 20.4  | H.4 Input boundaries, invalid values, and recovery states .....                         | 56 |
| 20.5  | H.5 Timeline, sensitivity, and graph-interaction coverage .....                         | 57 |
| 20.6  | H.6 Theme, document, export, and share coverage .....                                   | 57 |
| 20.7  | H.7 Importer and public-service evidence .....                                          | 58 |
| 20.8  | H.8 Desktop-package validation .....                                                    | 58 |
| 20.9  | H.9 Release-gate evidence and limitations .....                                         | 58 |
| 20.10 | H.10 Residual risk register .....                                                       | 59 |
| 21    | Appendix I — Alternatives considered and complete rationale ledger .....                | 60 |
| 21.1  | I.1 Problem-framing alternatives .....                                                  | 60 |
| 21.2  | I.2 Epidemiological-model alternatives .....                                            | 60 |
| 21.3  | I.3 Graph and spatial-data alternatives .....                                           | 61 |
| 21.4  | I.4 Intervention-algorithm alternatives .....                                           | 62 |
| 21.5  | I.5 Fairness and outcome alternatives .....                                             | 63 |
| 21.6  | I.6 User-experience and visualization alternatives .....                                | 63 |
| 21.7  | I.7 Persistence, export, and deployment alternatives .....                              | 64 |
| 21.8  | I.8 Documentation alternatives .....                                                    | 64 |
| 21.9  | I.9 Ledger conclusion .....                                                             | 65 |
| 22    | Appendix J — Worked mathematical case studies and audit calculations .....              | 66 |
| 22.1  | J.1 Case study 1: typed graph and baseline population audit .....                       | 66 |
| 22.2  | J.2 Case study 2: one source, one route, one daily exposure .....                       | 67 |
| 22.3  | J.3 Case study 3: multiple sources and the complement product .....                     | 68 |
| 22.4  | J.4 Case study 4: recovery and mortality ordering .....                                 | 68 |
| 22.5  | J.5 Case study 5: road closure as a graph transformation .....                          | 69 |
| 22.6  | J.6 Case study 6: budget feasibility with population constraints .....                  | 69 |
| 22.7  | J.7 Case study 7: degree and betweenness diverge .....                                  | 70 |
| 22.8  | J.8 Case study 8: Dijkstra transformation of a transmission route .....                 | 70 |
| 22.9  | J.9 Case study 9: a small max-flow/min-cut construction .....                           | 71 |
| 22.10 | J.10 Case study 10: Bayes update from symptom evidence .....                            | 71 |
| 22.11 | J.11 Case study 11: containment and tie-aware comparison .....                          | 72 |
| 22.12 | J.12 Case study 12: finite-difference sensitivity reading .....                         | 72 |
| 22.13 | J.13 Case-study conclusion .....                                                        | 72 |
| 23    | Appendix K — Complete module, data-flow, and review audit .....                         | 73 |

|       |                                                                                      |    |
|-------|--------------------------------------------------------------------------------------|----|
| 23.1  | K.1 Domain layer inventory .....                                                     | 73 |
| 23.2  | K.2 Scenario creation data flow .....                                                | 73 |
| 23.3  | K.3 Real-road import data flow and explicit semantic gap .....                       | 74 |
| 23.4  | K.4 Engine input/output audit .....                                                  | 74 |
| 23.5  | K.5 Planner input/output audit .....                                                 | 75 |
| 23.6  | K.6 Demonstration mode audit .....                                                   | 76 |
| 23.7  | K.7 Visualization and interaction audit .....                                        | 76 |
| 23.8  | K.8 Persistence, sharing, and local desktop audit .....                              | 77 |
| 23.9  | K.9 Explanation and export audit .....                                               | 77 |
| 23.10 | K.10 Build, package, and release audit .....                                         | 77 |
| 23.11 | K.11 Reviewer walkthrough .....                                                      | 78 |
| 23.12 | K.12 Final audit conclusion .....                                                    | 78 |
| 24    | Appendix L — Proof sketches, complexity bounds, and numerical design compendium .... | 80 |
| 24.1  | L.1 Proposition: population conservation is preserved .....                          | 80 |
| 24.2  | L.2 Proposition: elementary and aggregate exposure probabilities are bounded .....   | 80 |
| 24.3  | L.3 Expected value and variance of the binomial transition .....                     | 81 |
| 24.4  | L.4 Proposition: active intervention graph is a subgraph .....                       | 82 |
| 24.5  | L.5 Budget feasibility theorem for the common filter .....                           | 82 |
| 24.6  | L.6 Complexity of graph construction and one daily engine step .....                 | 83 |
| 24.7  | L.7 Complexity of Highest Degree and Random .....                                    | 83 |
| 24.8  | L.8 Complexity and correctness intuition of Brandes betweenness .....                | 84 |
| 24.9  | L.9 Dijkstra correctness under probability transformation .....                      | 84 |
| 24.10 | L.10 Max-flow/min-cut correctness intuition and cost .....                           | 85 |
| 24.11 | L.11 Finite-difference sensitivity and derivative interpretation .....               | 85 |
| 24.12 | L.12 Deterministic hash-trial properties .....                                       | 86 |
| 24.13 | L.13 Numerical safeguards and their formal role .....                                | 86 |
| 24.14 | L.14 What is proved and what is not .....                                            | 86 |
| 24.15 | L.15 Compendium conclusion .....                                                     | 87 |
| 25    | Appendix M — Reviewer traceability matrix and technical-defense guide .....          | 88 |
| 25.1  | M.1 Claim-to-evidence matrix .....                                                   | 88 |
| 25.2  | M.2 Requirement-to-module traceability .....                                         | 88 |
| 25.3  | M.3 Mathematical-defense questions and model answers .....                           | 89 |
| 25.4  | M.4 Engineering-defense questions and model answers .....                            | 90 |
| 25.5  | M.5 Formal review checklist .....                                                    | 91 |
| 25.6  | M.6 Presentation sequence for a 12–15 minute review .....                            | 91 |
| 25.7  | M.7 Final reviewer note .....                                                        | 92 |
| 26    | Glossary of terms and notation .....                                                 | 93 |

# 1 Executive summary

EpiGraph is a desktop-first academic simulator for examining epidemic containment on a finite city network. Its purpose is not to estimate a real epidemic. Instead, it makes several standard discrete-mathematics ideas inspectable in one reproducible environment: graph construction, weighted routes, centrality, flow cuts, probability updates, bounded optimization, and controlled comparison. A learner can create a synthetic city, import a deliberately small road network, select starting locations, define a limited intervention budget, and compare five established strategies under identical modeled conditions.

The central design decision was to prefer **transparent conditional simulation** over a visually impressive but untraceable forecast. Every output is therefore a **scenario-model projection** conditional on a graph, a parameter vector, a starting set, a seed, a duration, and a budget. The application labels this boundary in the user interface and retains it in persisted reports. This decision protects academic honesty: a road graph is neither a census nor a contact survey, and an algorithmically generated action is not public-health advice.

The system combines a React client, an Express/tRPC server, Drizzle-backed scenario persistence in the hosted application, and a local JSON persistence substitute in the portable desktop package. The shared TypeScript model is deliberately the source of truth for the graph, epidemic, budget, intervention, report, and fairness vocabulary. This reduced the risk that a chart, a report, and the simulator would silently disagree about fields or units.

The mathematical core is a discrete-time, graph-constrained **SIR-style** model. Each population-bearing vertex has susceptible, infected, recovered, and deceased counts. Road edges have a modeled transmission probability and a road-class capacity. Zero-population intersections are treated as connectors rather than as people. A deterministic keyed pseudo-random stream makes each node/day/event trial stable under all strategy runs, which is stronger than merely setting one global random seed. The same simulated random trial associated with a particular vertex, day, and event type is reused wherever the comparison conditions are otherwise identical.

Five strategies are contrasted: a seeded random baseline, highest degree, betweenness centrality, Dijkstra route blocking, and max-flow/min-cut. The project intentionally does **not** claim that one strategy is universally superior. Demonstration Mode contains five curated teaching scenarios in which each method is a unique winner once, and it explicitly explains the structural condition that makes that result plausible. A tie-aware comparator avoids falsely declaring the first item in a list—especially Random—to be a unique winner when outcomes are equal.

## 1.1 Principal contributions

| Contribution              | What was implemented and why it matters                                                                                                                                                                         |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reproducible city graph   | A seeded synthetic generator produces connected districts, facilities, intersections, roads, distances, probabilities, and capacities so that classroom experiments can be repeated exactly.                    |
| Fair five-way comparison  | All strategies receive the same graph, outbreak vector, initial infections, duration, budget, and deterministic keyed trials. This isolates intervention selection as the variable under study.                 |
| Interpretable mathematics | Each algorithm is represented by a direct, explainable objective rather than a black-box model: degree, shortest-path recurrence, flow capacity, or Bayes' rule.                                                |
| Bounded real-road route   | OpenStreetMap road geometry can be imported at neighbourhood scale with clipping, caps, cache, timeout, end-point failover, and a clear distinction between real geometry and synthetic epidemiological inputs. |
| Teaching support          | Guided scenarios, a timeline, sensitivity controls, a reset state, a presenter script, CSV/PDF export, and shareable reports turn the implementation into an assessable demonstration.                          |
| Portable delivery         | A Windows x64 portable Electron package embeds the required client runtime and curated road-graph fixtures so Demonstration Mode does not require a live map request.                                           |

## 1.2 Reading guide

Chapters 2–4 explain the problem framing, requirements, and architecture. Chapters 5–9 provide the formal graph model, state-transition equations, intervention algorithms, Bayesian scoring, and experiment protocol. Chapters 10–12 examine the interface, data path, desktop package, and verification evidence. Chapters 13–14 state limitations and conclusions. The decision register and appendices provide an auditable index of delivery decisions, demonstrations, tests, parameters, citations, and notation.

The report uses the word **modeled** carefully. “Modeled deaths,” for example, are output counts from a user-chosen daily transition probability; they are not measurements, clinical predictions, or casualty estimates. Similarly, an **OpenStreetMap (OSM)** road segment is genuine geometry,

but the app assigns a proxy population and a scenario transmission probability to make the graph useful for a classroom experiment.

## 2 Problem framing and requirements

### 2.1 Academic problem statement

The project began from a second-year discrete mathematics brief: use graph theory and probability to model how an outbreak might spread through a city and compare intervention points when resources are limited. The resulting question is formalized as follows. Given a finite graph  $G = (V, E)$ , an outbreak state over population-bearing vertices, and a budget  $B$ , choose a small action set  $A$  that modifies vertices or edges so that a chosen outcome measure is improved under a fixed scenario.

The important word is **choose**. A conventional epidemic animation only shows what happens after parameters are supplied. EpiGraph asks a more graph-theoretic question: **which limited roads or locations should a model select, and why?** The five strategies instantiate distinct answers. They are not novel algorithms; their educational value comes from comparing well-established techniques under one explicit protocol.

### 2.2 Requirements translated into engineering decisions

| Requirement                             | Decision and rationale                                                                                                                                                                |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synthetic city must work first          | A synthetic generator was placed ahead of live-data features. It guarantees a connected, small, repeatable graph and avoids blocking the lesson when a public service is unavailable. |
| Real-city roads should be possible      | The importer uses bounded neighbourhood road geometry rather than citywide networks. This protects responsiveness and makes the mathematical display readable.                        |
| Limited interventions                   | The shared budget contract caps road closures, quarantined nodes, and quarantined population. Strategies cannot “win” by implicitly closing the whole city.                           |
| Five techniques must be fairly compared | The runner stores shared fairness metadata and invokes the same epidemic engine for every plan. A keyed hash random stream stabilizes event draws.                                    |
| Decision-maker clarity                  | Recommendations expose the target, action, rationale, expected impact, and budget cost. Plain-English LLM text is optional explanatory narration, not a source of model results.      |

|                           |                                                                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Desktop-first delivery    | The layout is optimized for a laptop/desktop workspace and a portable Electron shell. Mobile-specific redesign was deliberately out of scope for the academic target. |
| Light and dark modes      | Global token-based theming was used rather than page-specific colors so the state is coherent across the simulator, reports, dialogs, and charts.                     |
| Demonstration reliability | Curated presets and offline fixtures are bundled because a live OpenStreetMap dependency can undermine a classroom presentation.                                      |

## 2.3 Non-goals and boundary decisions

The application does not collect, infer, or display clinical case records, personal movement traces, census populations, vaccine status, genomic data, or hospital capacity. It does not optimize a real intervention and it does not advise users to quarantine any actual location. Those exclusions are not omissions caused by lack of time alone; they are a design boundary. A realistic decision-support system would require domain experts, validated surveillance data, uncertainty calibration, governance, legal review, and operational safeguards far beyond this course project.

The model is also intentionally not agent based. An agent-based model could depict individual schedules and contacts, but it would multiply assumptions, obscure the discrete-mathematics focus, and make strategy fairness harder to audit in a compact demonstration. Aggregated location counts support direct binomial transitions and preserve visual clarity.

## 2.4 Success criteria

The release criteria were: the graph is connected and reproducible; all five strategies obey the same budget; every comparison has shared inputs; the map importer fails safely; reports preserve the scenario; visible controls are tested; the desktop build does not embed development-only Vite code; Demonstration Mode can obtain its curated graphs offline; and outcomes remain marked as academic scenario projections. The final quality gate reported TypeScript success and 62 passing automated tests across 24 test files, with one deliberately opt-in live OSM verifier outside the default suite.

# 3 Decision architecture: alternatives considered and selected rationale

## 3.1 Decision philosophy

The design used four recurring criteria: reproducibility, interpretability, boundedness, and pedagogical fit. A choice was preferred when it could be re-run from a small set of parameters,

explained to a second-year student, prevented a misleading extreme result, and left visible evidence for verification. This section records the major trade-offs. The detailed delivery ledger in Appendix A indexes every completed implementation item.

## 3.2 Major alternatives

| Decision area        | Alternatives considered                                                                  | Selected approach and why                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| City data            | Live citywide map; static synthetic graph; bounded live road graph                       | Synthetic graph is default; bounded OSM road geometry is optional. This yields a reliable baseline while preserving a tangible real-data extension.            |
| Epidemic structure   | Continuous differential equations; agent-based contacts; discrete aggregated node counts | Discrete node counts. The update steps can be inspected and the relation to binomial probability is explicit.                                                  |
| Randomness           | Fresh random draw per run; one sequential seed; keyed deterministic trials               | Keyed deterministic trials. The event identity controls the draw, so comparing a road closure does not shift unrelated random draws.                           |
| Intervention freedom | Unlimited closures; monetary objective; count-and-population caps                        | Explicit caps on roads, nodes, and population. This makes the phrase “under budget” mathematically enforceable without inventing questionable monetary values. |
| Strategy output      | Single winner only; multi-objective Pareto surface; ordered result with tie state        | Primary order by final infections then modeled deaths, with a tie state. It is simple for a course demo while avoiding a false unique winner.                  |
| Bayesian evidence    | Machine-learned classifier; explicit likelihood rule; no evidence model                  | Explicit Bayes update. Parameters are inspectable and evidence is demonstrative rather than claimed as diagnosis.                                              |
| 3D visualization     | Photorealistic city; abstract graph; miniature city                                      | An architectural miniature. It adds spatial texture while preserving visible graph relationships and intervention overlays.                                    |

|                  |                                                          |                                                                                                                                                        |
|------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Desktop delivery | Hosted web only; native rewrite; Electron portable shell | Electron portable shell with local loopback runtime. It reuses tested full-stack code and supports offline fixtures without a separate native rewrite. |
|------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|

### 3.3 Why the solution is modular

The shared domain model separates  $G$ , epidemic parameters  $\theta$ , budget  $B$ , evidence  $E$ , and actions  $A$ . This is not only a coding convention. It permits a precise causal reading of a comparison: holding  $(G, \theta, B, E)$  fixed, alter the rule that proposes  $A$  and observe the simulated consequences. The architecture then maps naturally to tests: graph tests validate  $G$ ; engine tests validate transitions; intervention tests validate  $A$  and  $B$ ; UI tests validate that a user can actually set and observe those variables.

### 3.4 Rejected complexity

Several ideas were consciously left out. No calibration routine fits parameters to observations; no uncertainty interval is labeled as a statistical confidence interval; no social graph is inferred; no dynamic travel demand is estimated; and no user data is sent to an epidemiological model. These omissions make the artifact narrower but more defensible. An academic simulator can be excellent by making its assumptions legible rather than by simulating a false level of operational realism.

## 4 System architecture and information flow

### 4.1 Logical architecture

EpiGraph is a full-stack TypeScript application. The browser client provides the five-step teaching workspace, 2D/3D graph explorers, timeline, sensitivity controls, charts, exports, theme selection, and Demonstration Mode. The server exposes typed tRPC procedures for scenario persistence, sharing, public-data import, and optional explanatory text. Shared modules implement the graph generator, epidemic engine, interventions, presets, and contracts. This arrangement keeps the algorithms independent from the visual layer and makes the same mathematical model available to server-side procedures, test code, and desktop runtime staging.

**Client workspace** → **typed server procedures** → **shared graph and simulation modules** → **persisted report / local desktop store**

## 4.2 Domain contract

The core contracts distinguish three kinds of vertex: population-bearing facilities such as homes, schools, hospitals, and offices; zero-population road intersections; and optional transit connectors. An edge contains identifiers, endpoints, a distance, a modeled transmission probability, a capacity, and a road class. A graph is therefore not a map image. It is a typed mathematical object with stable identifiers that can be serialized into a report or fixture.

The epidemic parameter vector is

$$\theta = (\alpha, \rho, \mu, T, I_0, \sigma),$$

where  $\alpha$  is the global transmission multiplier,  $\rho$  the per-day recovery probability,  $\mu$  the per-day mortality probability among people who remain infected after recovery draws,  $T$  the number of days,  $I_0$  the initial infected vertex set, and  $\sigma$  the simulation seed. The intervention budget is

$$B = (b_e, b_v, b_p),$$

where  $b_e$  limits road closures,  $b_v$  limits quarantined nodes, and  $b_p$  limits the sum of populations in quarantined nodes.

## 4.3 Portable runtime architecture

The Windows package launches Electron, starts a local Express server, and opens a Chromium window to the local loopback address. This design keeps front-end routing, server procedures, and desktop persistence behavior close to the hosted application. Production staging copies the built client and server runtime into the Electron package. The packaged application uses local file-based persistence instead of depending on the cloud database for saved desktop scenarios and reports.

The desktop boundary is intentionally explicit. Offline: synthetic graphs, all five algorithms, simulator computation, 2D/3D exploration, reports stored locally, CSV/PDF export, and fixture-backed Demonstration Mode. Online only: fresh OSM imports, LLM explanations, and public share links. This distinction was tested by inspecting the packaged client assets for stable curated-fixture identifiers rather than relying on an unminified TypeScript export name.

## 4.4 Trust and failure boundaries

The interface never assumes a public service will respond. OSM import has small radius defaults, an input cap, a cache, a per-endpoint timeout, bounded endpoint failover, an official OSM XML fallback, and named error states. The LLM explanation is separate from the simulation engine and receives a compact payload. Thus a failed external explanation or map fetch cannot change the underlying mathematical comparison; it only reduces an optional presentation capability.

## 5 Graph construction and data model

### 5.1 Formal graph representation

At any point, the simulator operates on an undirected graph  $G = (V, E)$ . The vertex set is divided into disjoint population-bearing locations  $V_P$  and movement connectors  $V_C$ . For every  $v \in V$ , the contract records an identifier, label, planar display position, type, district/block metadata, and population  $N_v \geq 0$ . A connector is characterized by  $N_v = 0$ . For every edge  $e = \{u, v\} \in E$ , the contract records distance  $d_e$ , modeled transmission probability  $\tau_e \in [0, 1]$ , capacity  $c_e$ , and road class  $r_e$ .

This partition is an important modeling choice. A city map contains junctions that are necessary for connectivity but do not themselves represent a building population. Treating every junction as an SIR compartment would artificially invent people. Removing them would destroy road paths. The implemented engine instead traverses zero-population connectors as path intermediates while applying population transitions only at  $V_P$ .

### 5.2 Synthetic generator

The default graph is a small connected synthetic city, rather than a random geometric graph with no narrative structure. A reproducible integer seed initializes a lightweight pseudo-random generator. The generator lays district hubs on a grid, places block intersections around each hub, attaches residences to each block, connects hub and intersection rings, attaches schools, hospitals, and offices to nearby intersections, and adds a bounded number of extra intersection roads. This constructs a graph with both local neighborhoods and cross-district corridors, allowing centrality and cuts to exhibit different behavior.

For a fixed configuration  $(D, B, H, S, H_o, O, \rho_G, \sigma)$ —districts, blocks/district, homes/block, schools, hospitals, offices, density, and seed—the output is fixed. Its stable identity incorporates the seed and main dimensions. Facility populations are generated synthetic values: homes use a bounded interval, facilities use a type-specific central value with controlled multiplicative variation. This is a pedagogical model of heterogeneous locations, not a demographic reconstruction.

The Euclidean display distance between two synthetic positions is

$$d_{u,v} = \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2},$$

and the reported route distance is  $d_e = \max\left(0.12, \frac{d_{u,v}}{9.5}\right)$  km, rounded by the implementation to two decimal places. Long synthetic edges are classified primary or secondary; short edges local. The road class determines a capacity proxy: primary 1200, secondary 700, local 250, and otherwise 80. It also enables a consistent visual language in the 2D and miniature 3D renderers.

The configured road probability is sampled in a visible bounded interval:

$$\tau_e = \text{round}_3(\tau_{\min} + U_e \max(0, \tau_{\max} - \tau_{\min})),$$

where  $U_e$  is a seeded uniform deviate and  $\text{round}_3$  rounds to three decimal places. The explicit minimum and maximum sliders exist so a teacher can make a network weakly or strongly coupled without editing code.

### 5.3 Imported road geometry

The real-road route uses the **Overpass API** first and an official OSM small-area XML route as fallback. An import configuration contains a place label, centre latitude/longitude, radius, maximum nodes, and a footway flag. Coordinates are clamped to valid latitude and longitude ranges, the radius to  $[0.05, 1.5]$  km, and the maximum nodes to  $[20, 320]$ . The defaults are intentionally neighbourhood scale. The application caches normalized imports for 15 minutes under a key based on rounded centre, radius, cap, and footway flag.

Haversine distance is used to crop road geometry and measure imported edges:

$$a = \sin^2\left(\Delta\frac{\varphi}{2}\right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2\left(\Delta\frac{\lambda}{2}\right), \quad d = 2R \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right),$$

with  $R = 6371$  km. A way crossing the selected boundary is clipped before node-limit checking. The normalization then removes duplicate edge fragments, rejects fewer than eight nodes or seven edges, rescales coordinates for visualization, and preserves OSM IDs and attribution. Road type is converted to a model class. For example, primary roads receive  $(\tau_e, c_e) = (0.061, 150)$  and local roads  $(0.034, 65)$  in the imported graph normalizer.

Crucially, a junction population is a documented proxy  $N_v = 45 + 28 \deg(v)$  for imported roads, not population data from OSM. This is a deliberate compromise: the graph needs count-bearing locations to run the same SIR engine, but road geometry does not supply occupancy. The report therefore never describes imported simulation outcomes as real city forecasts.

### 5.4 Actions as graph transformations

An intervention action transforms graph availability, not the original serialized graph. A road closure contributes an identifier to  $E_{\text{closed}}$ ; a quarantine contributes an identifier to  $V_Q$ . The active edge set on day  $t$  is

$$E_t^A = \{e = \{u, v\} \in E : e \notin E_{\text{closed}}, u \notin V_Q, v \notin V_Q\}.$$

This non-destructive representation allows the baseline and all alternatives to share one original graph. It also allows a Reset control to clear manual transformations and restore the teaching scenario without rebuilding the city differently.

## 6 Discrete epidemic mathematics

### 6.1 Relationship to classical SIR

Kermack and McKendrick's classical epidemic formulation models rates of change in susceptible, infected, and removed compartments [1]. In its familiar homogeneous form,

$$d\frac{S}{N}t = -\beta S \frac{I}{N}, \quad d\frac{I}{N}t = \beta S \frac{I}{N} - \gamma I, \quad d\frac{R}{N}t = \gamma I.$$

Those differential equations motivate the state names, but they are not the equation executed by EpiGraph. A homogeneous well-mixed  $\beta$  is unsuitable for showing how roads, bridges, and cuts change access between locations. The application instead uses a finite daily update on each population-bearing vertex. This distinction matters:  $\alpha$ ,  $\rho$ , and  $\mu$  are scenario probabilities or multipliers, not fitted clinical rates.

At day  $t$ , vertex  $v$  has count vector

$$X_{v(t)} = (S_{v(t)}, I_{v(t)}, R_{v(t)}, D_{v(t)}),$$

where  $S_v + I_v + R_v + D_v = N_v$ . An initial selected population location begins with

$$I_{v(0)} = \min(12, N_v), \quad S_{v(0)} = N_v - I_{v(0)},$$

and all other initial counts are zero. If no initial location is selected, the largest population-bearing vertex becomes the deterministic fallback. This avoids a silent empty epidemic while ensuring that every comparison starts at the same location.

## 6.2 Edge exposure and path transmission

For an active road  $e$  leaving infectious source  $u$ , the per-contact daily probability is

$$x_e = \min(0.95, \alpha\tau_e).$$

If  $I_{u(t)}$  infectious people each contribute one simplified independent opportunity across that road, the probability of at least one opportunity is the complement of zero opportunities:

$$q_{e(t)} = 1 - (1 - x_e)^{I_{u(t)}}.$$

The cap at 0.95 prevents a supplied multiplier from turning a transparent scenario probability into an invalid or numerically extreme probability. It is not an empirical biological ceiling.

For a route  $\pi = (e_1, e_2, \dots, e_k)$  through zero-population connectors, EpiGraph uses the sequential-product approximation

$$q_{\pi(t)} = \prod_{\ell=1}^k q_{e_\ell}(t).$$

The connector search retains the strongest route probability to a connector and visits it again only if that value increases. This suppresses cyclic expansion while keeping useful roadway connectivity. Multiple contributing source paths combine at a destination  $v$  with a no-exposure complement:

$$p_{v(t)} = 1 - \prod_{u \in U_v} (1 - q_{u \rightarrow v}(t)).$$

This aggregation makes a location's probability increase when it has multiple modeled infectious sources without simply summing probabilities beyond 1.

## 6.3 Binomial daily transitions

The engine samples each count transition by repeated deterministic Bernoulli trials. For integer  $n$  and probability  $p$ , a binomial variable obeys

$$P(Y = k) = \text{choose}(n, k) p^k (1-p)^{n-k}, \quad k = 0, \dots, n,$$

with expected value  $E[Y] = np$  and variance  $np(1-p)$ . The implementation generates one reproducible Bernoulli decision for every index  $j \in \{0, \dots, n-1\}$ , rather than invoking an opaque external random distribution.

Given recovery probability  $\rho$  and mortality probability  $\mu$ , the daily values are

$$\begin{aligned} R_v^+(t) &\sim \text{Binomial}(I_{v(t)}, \rho), \\ D_v^+(t) &\sim \text{Binomial}(I_{v(t)} - R_v^+(t), \mu), \\ I_v^+(t) &\sim \text{Binomial}(S_{v(t)}, p_{v(t)}). \end{aligned}$$

The update is then

$$\begin{aligned} S_{v(t+1)} &= S_{v(t)} - I_v^+(t), \\ I_{v(t+1)} &= I_{v(t)} - R_v^+(t) - D_v^+(t) + I_v^+(t), \\ R_{v(t+1)} &= R_{v(t)} + R_v^+(t), \quad D_{v(t+1)} = D_{v(t)} + D_v^+(t). \end{aligned}$$

Recovery is drawn before mortality, so  $\mu$  operates on people who remain infected that day. The state sum is conserved: adding the four right-hand sides cancels each transition, giving  $N_{v(t+1)} = N_{v(t)}$ . This conservation identity is a useful unit-test invariant.

## 6.4 Determinism and common random numbers

An ordinary sequential random generator can make a comparison subtly unfair. If Strategy A removes an edge and thereby changes how many earlier random calls occur, later events can receive different random draws even at unchanged locations. EpiGraph uses a keyed hash  $U(\sigma, \text{key}, j)$  in  $[0, 1)$ , where the key includes event type, vertex, and day. A recovery at node  $v$  on day 7 always consults the same sequence  $U(\sigma, \text{recovery:v:day:7}, j)$  in every run.

This is a common-random-numbers design. It does not remove stochasticity from a scenario, but it cancels needless random variation when comparing interventions. The deterministic trial method is preserved as `keyed_hash` in fairness metadata.

## 6.5 Derived measures and interpretation

The interface reports aggregate metrics by summing vertex counts. Cumulative infections are  $C(t) = \sum_{v \in V_P} (N_v - S_{v(t)})$ . Peak active cases are  $\max_t \sum_v I_{v(t)}$ , outbreak duration is the last  $t$  with active infections, and modeled mortality is  $\sum_v D_{v(T)}$ . A didactic effective contact proxy could be written  $R_{\text{local}} \approx \alpha \tau_e \frac{I_u}{S_u}$  under simplified conditions, but EpiGraph does not label such a proxy as a real-world  $R_0$ . The network is heterogeneous, interventions change availability, and parameters are not calibrated; reporting a clinical reproduction number would be unwarranted.

# 7 Intervention algorithms and constrained optimization

## 7.1 Feasible action set

Every strategy produces a ranked candidate list, but the budget filter chooses only feasible actions. For an action set  $A$ , define  $E_A$  as closed roads and  $V_A$  as quarantined population-bearing nodes. The feasible set is

$$F_B = \left\{ A : |E_A| \leq b_e, |V_A| \leq b_v, \sum_{v \in V_A} N_v \leq b_p \right\}.$$

The code selects node actions first in candidate order, skipping a candidate whose population would exceed  $b_p$ , then selects edge actions until  $b_e$  is reached. This lexical procedure is straightforward and inspectable. It is not presented as a globally optimal mixed integer program; the project compares heuristics under a shared constrained rule.

## 7.2 Random baseline

The Random strategy shuffles population-bearing nodes and roads using a seed  $\sigma + 101$ . It exists as a baseline, not as a recommended policy. Including it is methodologically valuable: an informed strategy should be judged against a reproducible uninformed allocation, and Demonstration Mode can show that chance can occasionally align with a useful corridor. A Random unique win is explained as a seeded scenario result, never a universal claim.

## 7.3 Highest degree

For an undirected graph, degree is

$$d(v) = |\{e \in E : v \in e\}|.$$

The strategy orders vertices by decreasing  $d(v)$ . An edge receives the sum of its endpoint degrees,

$$s_{d(\{u,v\})} = d(u) + d(v),$$

and roads are ordered by decreasing score. The idea is that a high-degree location offers many direct contact opportunities; reducing access there can be strong when local hubs dominate the paths. It can fail when a moderate-degree bridge separates larger regions, which motivates betweenness.

## 7.4 Betweenness centrality

Betweenness measures how often a vertex lies on shortest paths between other pairs:

$$C_{B(v)} = \sum_{s \neq v \neq t} \sigma_{s,t} \frac{v}{\sigma_{s,t}},$$

where  $\sigma_{s,t}$  is the number of shortest paths from  $s$  to  $t$  and  $\sigma_{s,t}(v)$  the number passing through  $v$ . The implementation follows Brandes' unweighted algorithm [4]. For each source it performs breadth-first discovery, records predecessor lists and path counts, then back-propagates dependencies. Since the graph is undirected, final scores are divided by two to avoid double counting ordered pairs.

Brandes reports  $O(nm)$  time for unweighted networks and  $O(nm + n^2 \log n)$  for weighted networks, using  $O(n + m)$  space [4]. At the project's capped teaching scale, the unweighted version is transparent and sufficiently responsive. Edges are ranked by the sum of their endpoint vertex scores. The chosen semantics are “bridge-like locations frequently on topological shortest paths,” not “statistically estimated movement corridors.”

## 7.5 Dijkstra route blocking

Transmission probability is converted into an additive nonnegative edge cost:

$$w_e = -\ln(\max(0.0001, \min(0.9999, \tau_e))).$$

For a path  $\pi$ , the total cost is

$$\sum_{e \in \pi} w_e = -\ln\left(\prod_{e \in \pi} \tau_e\right).$$

Because  $-\ln(x)$  is strictly decreasing on  $(0, 1]$ , minimizing the transformed path cost is equivalent to maximizing the product of edge probabilities. This is the mathematical justification for using Dijkstra's algorithm on nonnegative weights [6]. The strategy computes routes from initial outbreak locations to the five largest non-infected population destinations, counts how frequently each road appears, and ranks roads by that count. Endpoint occurrence counts induce a node ranking. It therefore targets repeatedly likely source-to-major-destination routes, not merely the nearest road.

## 7.6 Max-flow/min-cut

Each road receives a capacity  $c(e)$  derived from its class or imported class proxy. The graph is represented symmetrically in a residual network. Starting at the initial infected vertex, the implementation finds augmenting paths by breadth-first search, subtracts bottleneck capacity, adds reverse capacity, and continues until no augmenting path reaches the selected target. The target prioritizes a hospital if available, otherwise the largest population-bearing destination.

For a cut partition  $(S, T)$  with source in  $S$  and target in  $T$ , capacity is

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v).$$

The max-flow/min-cut theorem equates the maximum achievable flow with the minimum cut capacity [5]. After augmentations, vertices reachable from the source in the residual network form  $S$ ; original roads crossing from reachable to non-reachable vertices are min-cut candidates. This strategy is most interpretable when a small capacity-limited separation disconnects source and destination regions. Capacities are a structural proxy, not observed passenger counts.

## 7.7 Comparison objective

For every feasible plan, the engine evaluates a shared baseline cumulative infection total  $C_0$  and computes containment

$$\eta(A) = \text{clip}_{0,1}\left(\frac{C_0 - C_A}{C_0}\right).$$

The declared primary comparison order is lower final cumulative infections, then lower modeled mortality. The system also reports peak active infections, peak day, duration, budget use, components after intervention, actions, and hotspot scores. This keeps the winner rule concise while leaving the learner able to inspect trade-offs rather than treating a single scalar as the only value judgment.

## 8 Bayesian hotspot scoring and actionable recommendations

### 8.1 Why a Bayesian component was included

Graph intervention identifies where the topology matters; it does not, by itself, encode the user's stated local evidence. The application therefore includes a lightweight Bayesian hotspot module. It exposes a prior, an evidence likelihood, a false-positive likelihood, and a posterior for each population-bearing vertex. The aim is explanatory: a student can see how a prior is revised. It is **not** a diagnostic classifier; the likelihood parameters are illustrative inputs, not estimated clinical sensitivities or prevalence values.

### 8.2 Prior construction

For a location  $v$ , the engine receives a modeled infection probability  $p_v$  and forms an exposure quantity  $h_v$  from connected infectious sources. The implementation accumulates a road probability, infectious count, and source-population ratio for each available incident opportunity, excluding zero-population source vertices. The clamped prior is shown below, where  $C_{a,b}(x)$  means “clamp  $x$  to the interval from  $a$  to  $b$ .”

$$P(H_v) = C_{(0.001, 0.97)} [ 0.015 + 0.55 p_v + 0.65 h_v ]$$

The offset 0.015 prevents a connected but low-exposure node from becoming logically impossible; the 0.55 and 0.65 coefficients visibly combine current modeled state and surrounding exposure. They are deliberately **not** fitted epidemiological coefficients. This is a decision-support teaching formula, not a medical claim.

### 8.3 Evidence likelihoods and posterior

If the user enters  $a_v$  symptomatic observations out of  $n_v$  observations and an evidence-strength control  $k_v$  between zero and one, the engine uses the following transparent calculation.

$$\begin{aligned} z_v &= C_{(0, 1)} [ a_v / \max(1, n_v) ] \\ L_v &= C_{(0.05, 0.96)} [ 0.18 + 0.65 z_v k_v ] \\ F_v &= C_{(0.01, 0.42)} [ 0.04 + 0.12 z_v k_v ] \\ P(H_v | E_v) &= L_v P(H_v) / [ L_v P(H_v) + F_v (1 - P(H_v)) ] \end{aligned}$$

Here  $H_v$  denotes the model's “hotspot” hypothesis,  $E_v$  denotes the entered evidence,  $L_v$  is the illustrative likelihood of that evidence under the hotspot hypothesis, and  $F_v$  is its illustrative false-positive likelihood. The posterior is clamped to the unit interval, sorted descending, and assigned a rank. When no symptom evidence is supplied, the system retains the network-derived prior and explicitly states that evidence was absent. When evidence is present, the output records the observed ratio  $\frac{a_v}{n_v}$ .

## 8.4 Interpretation discipline

A large posterior means only that the supplied scenario, evidence-strength setting, and illustrative likelihood rule elevate a **model hotspot score**. It does not establish disease presence, diagnostic sensitivity, specificity, medical prevalence, or clinical validity. The responsible wording is therefore: “under the supplied scenario and symptom evidence, this location is ranked for attention.”

## 8.5 Recommendations

The recommendation generator takes the first five actions from the winning plan and enriches every action with a target, an action label, a rationale, an expected-impact statement, and a budget cost. For a road it averages available endpoint hotspot scores; for a node it presents that node’s score and population cost. The expected-impact text remains conditional on the chosen strategy and scenario assumptions. This decision fulfills the usability requirement to convert a mathematical output into a comprehensible action without turning the application into real-world public-health advice.

# 9 Experimentation, fairness, sensitivity, and demonstration protocol

## 9.1 Fair comparison protocol

A strategy comparison is defensible only if selection rule is the variable changing. For all five strategies EpiGraph holds fixed

$$(G, \theta, B, E, I_0, T, \sigma),$$

where  $G$  is the graph,  $\theta$  epidemic parameters,  $B$  budget,  $E$  symptom evidence,  $I_0$  initial infections,  $T$  duration, and  $\sigma$  seed. The runner first computes a no-algorithm baseline, then creates a plan for each strategy, runs the same discrete-time engine, and retains fairness metadata: graph ID/fingerprint, initial infected IDs, full parameter vector, budget, seed, and deterministic-trial method.

```
baseline = runDiscreteTimeSIR(graph, parameters)
for strategy in [random, degree, betweenness, dijkstra, minCut]:
    plan = planForStrategy(graph, parameters, budget, strategy)
    outcome = runDiscreteTimeSIR(graph, parameters, plan.actions)
    winner = lexicographicMin(outcomes, finalInfected, modeledDeaths)
if moreThanOneOutcomeHasWinnerMetrics: status = "tied"
```

The use of a baseline is also necessary for containment:  $\eta = 0$  when  $C_0 = 0$ , avoiding division by zero. A tied result is surfaced as inconclusive rather than assigning a winner by array order. This repair was particularly important because a naïve first-item sort could create an embarrassing false impression that Random had won.

## 9.2 Sensitivity analysis

The sensitivity panel varies transmission  $\alpha$ , recovery  $\rho$ , mortality  $\mu$ , and strategies across bounded ranges. Its interpretation is directional, not a calibrated statistical interval. For a reported output  $Y(\theta)$ , a local central finite-difference sensitivity can be defined as

$$\partial Y / \partial \theta_j \approx [Y(\theta + \delta e_j) - Y(\theta - \delta e_j)] / (2\delta) \quad (26)$$

For a scale-free comparison, an elasticity is

$$E_j = (\theta_j / Y) (\partial Y / \partial \theta_j)$$

when  $Y \neq 0$ . These derivatives explain what a sweep visually conveys: a positive derivative with respect to  $\alpha$  suggests stronger modeled transmission raises the selected output locally; a negative derivative with respect to  $\rho$  suggests faster modeled recovery lowers it. The app does not call these causal estimates outside its declared rules.

## 9.3 Curated Demonstration Mode

Five named presets use two bounded real-road fixtures: East Village, New York and Shoreditch, London. The original presets were searched and verified against bounded road graphs; static fixture copies then made the teaching sequence reproducible without a live request. The table preserves exact release parameters.

| Preset                | Claimed winner   | $\alpha, \rho, \mu, T, \sigma$ | Budget and verified outcome                                     |
|-----------------------|------------------|--------------------------------|-----------------------------------------------------------------|
| Chance alignment      | Random           | 0.22, 0.06, 0.006, 16, 0.28    | $b_p = 1$ , 3,264 final infections; 140 modeled deaths          |
| Hub protection        | Highest degree   | 0.18, 0.06, 0.006, 16, 0.28    | $b_p = 1$ , 3,264 final infections; 167 modeled deaths          |
| Bridge containment    | Betweenness      | 0.26, 0.08, 0.012, 20, 0.41    | $b_p = 1$ , 14,402 final infections; 932 modeled deaths         |
| Likely-route blocking | Dijkstra         | 0.18, 0.06, 0.006, 16, 0.41    | $b_p = 1$ , $b_p = 1800$ , 12 final infections; 1 modeled death |
| Minimum cut           | Max-flow/min-cut | 0.18, 0.06, 0.006, 16, 0.26    | $b_p = 1$ , 2,082 final infections; 147 modeled deaths          |

These are not benchmark claims that the named strategy dominates outside its fixture. Each scene exists to demonstrate a structural lesson: chance can align; hubs can dominate; bridges can matter more than degree; a likely route can be blocked; or a small cut can separate a source region. The live importer test is opt-in because public data can change or be unavailable; fixture verification is the routine reproducibility control.

## 9.4 Manual what-if mode

Manual closures and quarantines are deliberately kept separate from the five-way comparison. They let a user pose “what if this road is closed?” without contaminating the algorithm-selected action sets. The manual runner computes a new baseline and then evaluates the user actions through the same engine. This preserves a clean distinction between an algorithm comparison and exploratory scenario play.

# 10 Interface, visualization, reports, and exports

## 10.1 Workspace design

The workspace was refactored from a crowded single-page control area into an explicit learning flow: Setup, Network, Outcomes, Comparison, and Analysis. The top ribbon does not imply that the mathematics occurs in a fixed sequence; it gives the learner a navigable mental model. Setup establishes graph and epidemic assumptions. Network exposes the geometry and selected starting locations. Outcomes shows the day-by-day trajectory. Comparison makes the fair five-way result legible. Analysis provides Bayesian hotspots, manual what-if actions, and sensitivity sweeps. Each quantitative input is accompanied by a definition, a permitted range, and a plain-language effect. This is a design decision with mathematical significance: a user should not be able to type a mortality rate of 300% or a graph size that will make the browser unresponsive without receiving a bounded value or clear feedback. Boundary clamping is part of model integrity, not merely UI polish.

## 10.2 2D network explorer

The 2D explorer represents vertices by facility type and epidemic state, edges by road class, and interventions by visible closure/quarantine treatment. It supports zoom, pan, click selection, reset, and legends. A selection cap of three population-bearing initial-infection locations is intentional. It provides meaningful multi-source scenarios while keeping causal explanation manageable. Clicking a fourth candidate produces feedback rather than silently dropping a selection. The timeline renders snapshots  $t = 0, \dots, T$  and has previous/next buttons, a scrubber, and play/pause. Animation is subordinate to state inspection; the app respects reduced-motion preference and retains static controls. The state palette deliberately differentiates susceptible, infected, recovered, deceased, and quarantined locations with distinct hues rather than relying only on opacity or animation.

## 10.3 Architectural miniature 3D view

The 3D view is not a second simulation. It reads the same graph and snapshot contracts as the 2D view. The renderer uses varied building footprints and rooflines, facility-specific silhouettes, road surfaces with primary/secondary markings, block parcels, green buffers, district lighting, outbreak halos, quarantine fencing, and a tighter tabletop camera. The intent was a “high-detail

architectural miniature”: richer than abstract nodes and lines, but not a photorealistic claim about a real street.

This visual choice resolves a tension. An abstract graph supports mathematical reading but can look like a disconnected assignment. A photorealistic city can imply false realism and obscure network structure. The miniature retains topology: roads remain visually traceable, facilities are distinguishable, and intervention barriers are visible. Tests isolate building-profile and facility-geometry rules in a pure model module so visual refactoring does not alter epidemic logic.

## 10.4 Results, sharing, and downloads

The Outcome view separates time-series charts from the comparison scorecard. The comparison table exposes cumulative infections, active cases, recoveries, modeled deaths, peak metrics, budget use, and recommended actions. CSV export provides current tabular strategy results. The PDF export generates a report containing graph snapshot, configuration, interventions, charts, recommendations, and optional LLM explanation; user feedback exists for export failure. Persisted report payloads can create a public share page with equivalent sections.

The LLM explanation is intentionally downstream. It summarizes the already computed winner and critical locations in plain English for a non-technical decision-maker. The compact payload repair ensured normal scenarios fit the server input limit. The model never authorizes the strategy, computes the SIR timeline, or replaces the explicit recommendation rationale; its failure cannot alter the numeric result.

## 10.5 Theme and interaction quality

Light/dark mode is global and persistent. It is implemented through shared tokens rather than component-local inversion, preventing common failures such as invisible text on an otherwise dark card. Interaction tests cover theme persistence and document-class cleanup. Motion durations remain short and can be disabled. While mobile-specific adaptation and specialized assistive-technology extensions were outside the scoped deliverable, keyboard reachability, focus behavior, error visibility, and reduced motion were treated as core quality requirements rather than decorative extras.

# 11 Data provenance, governance, and desktop packaging

## 11.1 Road geometry versus model variables

OpenStreetMap contributes road geometry, labels, classes, and identifiers under its published attribution and licence terms [8]. The app preserves an attribution string in the normalized graph. It does not source people, symptoms, infections, deaths, or travel volumes from OSM. Every such quantity used by the simulation is synthetic or user-configured. This distinction is repeated

in import guidance, generated reports, and portable-package documentation because a realistic-looking street map can otherwise create an unwarranted impression of real epidemiology. The primary public-data route uses bounded Overpass queries [9]. The query filters road types and a radius around a selected point. The implementation performs one attempt per public endpoint instead of unbounded retries. Each attempt has an eight-second timeout, while the overall import orchestration has a 28-second deadline. If public interpreters fail, the app requests a bounded official OSM XML map extraction [10]. Failure is represented as a named status such as upstream unavailable, oversized network, invalid area, or empty network. The visible workspace error state remains on screen so a learner can recover by reducing the area.

## 11.2 Import safety measures

| Measure                                      | Reason it was selected                                                                                                                                         |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Radius clamp: 0.05–1.5 km                    | Limits query scale and supports a network that can be meaningfully read in a class session.                                                                    |
| Node clamp: 20–320                           | Keeps graph algorithms, WebGL rendering, and comparison UI responsive; checks happen before normalization.                                                     |
| Boundary clipping                            | A way may contain full geometry beyond the chosen circle. Clipping before the cap prevents an adjacent long road from causing a misleading oversize rejection. |
| Cache: 15 minutes                            | Avoids repeatedly asking public services for the identical neighborhood and improves repeat use.                                                               |
| Multiple endpoints and one official fallback | Public availability varies; bounded failover improves resilience without hammering third-party infrastructure.                                                 |
| Attribution and proxy flag                   | Makes provenance explicit and records that modeled junction population is not source data.                                                                     |
| Offline fixtures for demos                   | Protects the teaching path from public-service availability, rate limits, and mutable upstream geometry.                                                       |

## 11.3 Portable Windows package

Electron Builder produces a portable Windows x64 executable rather than a traditional installer. The portable executable is suitable for copying to a writable directory and launching directly. Local scenario/report data is stored beside the executable; an unsigned-file warning may require normal Windows user approval. The packaged runtime deliberately avoids statically importing Vite development plugins. This resolved a production launch failure in which server code accidentally captured JSX/development configuration that should not exist in a runtime bundle. Demonstration Mode initially depended on live OSM, which created a user-visible “map could not load” failure in a setting where demonstrations should be reliable. The repair moves the curated

East Village and Shoreditch graph snapshots into fixture JSON files imported by a shared module. The client calls the fixture loader for named presets rather than making a live request. Package validation searched for stable scenario identifiers and fixture data in minified client assets, not merely a TypeScript export name that a bundler is free to erase. The final artifact was copied with a verified SHA-256 checksum and has a separately pending user-device smoke confirmation.

## 11.4 Security and privacy observations

The hosted application uses managed authentication and database infrastructure for persistence. The portable desktop variant uses local persistence and does not require a user account to run core simulations. No secret is hard-coded into the mathematical engine. External connection failure is handled as a degraded capability, not a silent fallback that changes the model. Public share links should be treated as intentionally published scenario reports; they are not a substitute for sensitive-data storage governance.

# 12 Verification, validation, and quality assurance

## 12.1 What was verified

Verification asks whether the software implements its stated contracts. Validation asks whether those contracts are appropriate for the intended educational use. The project distinguishes these two. Automated tests can establish reproducibility, budget enforcement, state conservation, route selection behavior, and UI callbacks. They cannot establish that the scenario predicts a real outbreak. The latter is deliberately not claimed.

The release gate reported successful TypeScript checking and 62 passing automated tests across 24 test files. The quality-assured test suite covers graph generation; probability bounds; SIR transitions; budget enforcement; strategy fairness; Bayesian scoring; recommendations; OSM parser/normalizer/fallbacks; report persistence; public sharing; PDF export behavior; demonstration preset reproducibility; reset state; CSV export; theme persistence; graph pan/zoom/selection; timeline bounds; sensitivity controls; and desktop-runtime boundaries. One live OSM reproduction test remains opt-in because it intentionally depends on a third-party mutable network service.

## 12.2 Testable invariants

| Invariant                     | Test significance                                                                            |
|-------------------------------|----------------------------------------------------------------------------------------------|
| $S_v + I_v + R_v + D_v = N_v$ | The local count conservation identity detects an invalid state update or double subtraction. |
| $0 \leq p_{v(t)} \leq 1$      | All infection probabilities are clamped, preventing invalid binomial trials.                 |

|                                          |                   |                                                                                                                       |
|------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| $ E_A  \leq b_e,$<br>$\sum N_v \leq b_p$ | $ V_A  \leq b_v,$ | No strategy can exceed the stated intervention resource constraints.                                                  |
| Common $G, \theta, B, I_0, T, \sigma$    |                   | Fairness metadata and deterministic event keys prove the intended common-condition comparison.                        |
| Tie status                               |                   | Equal primary/secondary outcome metrics must produce tied, not a falsely unique first strategy.                       |
| Imported graph cap                       |                   | The 320-node cap is aligned in browser validation, server validation, normalizer, and contract tests.                 |
| Offline preset access                    |                   | Fixture-backed preset selection is covered without a live map call and package assets are checked for stable content. |

## 12.3 UI interaction evidence

Rendered regression tests went beyond static component rendering. They exercise editable input boundaries, source-mode switches, importer pending and error states, manual interventions, timeline previous/next limits, sensitivity parameter changes and playback, global theme behavior, 2D pan/zoom/selection, reset actions, CSV download, and Demonstration Mode previous/next/load/close callbacks. This matters because a mathematically correct function is not sufficient if the surrounding controls can leave it unreachable or send stale values.

## 12.4 Visual and runtime QA

The desktop interface received visual review for layout, dark/light contrast, 3D readability, and demonstration controls. The 3D renderer’s rendering logic was separated from the model profile rules to preserve testability. The Electron package was checked for absence of a Vite plugin reference and for the presence of offline fixture content. A final Windows device should still launch a curated demo preset: this external user-side smoke check is the remaining confirmation listed in the release checklist, and it is honestly recorded as pending rather than silently assumed.

## 12.5 Quality limitations

Test success is not proof of absence of all defects. Browser graphics drivers, Windows security policy, third-party map availability, and user-specific file permissions remain outside the sandbox test environment. The test suite specifically treats public map availability as a recoverable external boundary. The fixture path reduces—but cannot mathematically eliminate—risk in the portable classroom demo because it depends on the complete packaged runtime functioning on a user’s Windows system.

## 13 Limitations, responsible interpretation, and future work

### 13.1 Mathematical limitations

The model assumes daily discrete time, fixed node populations, independent Bernoulli opportunities, simple recovery/mortality probabilities, no reinfection, and static road availability except for selected interventions. It does not include latent exposure, heterogeneous susceptibility, household mixing, age structure, vaccination, seasonality, mobility schedules, health-system feedback, behavioral adaptation, or endogenous intervention compliance. The path product  $\prod q_e$  is a transparent approximation, not a derivation from contact-tracing data.

The graph is undirected even where a real road may be one-way, because the primary educational goal is connectivity and algorithm comparison. Imported OSM oneway metadata is retained but does not turn the simulation into a directed travel model. Capacity is a road-class proxy for the min-cut lesson, not vehicle flow, pedestrian throughput, or transmission carrying capacity. The primary winner objective treats final infections then modeled deaths lexicographically; a real decision would require values, constraints, costs, equity measures, and stakeholder governance beyond the scope of this implementation.

### 13.2 Statistical limitations

The pseudo-random simulation uses a single deterministic scenario realization. Common random numbers make **relative** comparison less noisy, but they do not quantify uncertainty. There is no Monte Carlo ensemble, posterior distribution over parameters, confidence interval, or calibration to observations. A next research step could run  $M$  independent seeds and report sample means and intervals, e.g.

$$\begin{aligned}\bar{Y} &= (1/M) \sum_{m=1}^M Y_m \\ SE(\bar{Y}) &= \sqrt{[1 / (M(M-1))] \sum_{m=1}^M (Y_m - \bar{Y})^2}\end{aligned}$$

but such an extension must then explain multiple-comparison effects and the distinction between simulation variation and real-world uncertainty.

### 13.3 Ethical limitations

The project purposefully rejects deterministic labels such as “this school must close.” Recommendations are scenario outputs under assumptions, and the app describes them as such. Quarantine and road closure are socially costly actions that require legal authority, public participation, proportionality, accessibility, and equity analysis in real use. A graph algorithm cannot establish any of these conditions.

The risk scoring feature is especially constrained. Symptom evidence may be incomplete, biased, and nonspecific. The prior and likelihood coefficients are illustrative. Outputs should never be used to identify or stigmatize people, neighborhoods, facilities, or communities. The most

defensible classroom interpretation is that Bayes' rule changes a numerical belief when evidence assumptions change.

### 13.4 Future work, ordered by responsible value

| Extension                                 | Precondition before it would be responsible                                                                                       |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Monte Carlo confidence summaries          | Document independent-seed protocol and distinguish simulation variability from real-world inference.                              |
| Directed and time-varying mobility        | Use a clearly licensed, aggregate source and document how travel assumptions are generated.                                       |
| Richer compartments such as SEIR          | State assumptions for latency and infectiousness and add conservation and transition tests.                                       |
| Multi-objective intervention optimization | Engage stakeholders to justify weights for infections, deaths, economic cost, access, and equity rather than hiding them in code. |
| Accessibility expansion                   | Conduct user testing with assistive-technology users instead of claiming compliance from static checks.                           |
| Authenticated data workflows              | Establish privacy, retention, authorization, audit, and incident-response policy before accepting sensitive information.          |

## 14 Conclusion

EpiGraph demonstrates that a strong discrete-mathematics project need not invent a novel algorithm to be technically substantial. Its contribution is the disciplined integration of established ideas: a finite city graph, bounded probability transitions, centrality, transformed shortest paths, minimum cuts, Bayes' rule, constrained actions, deterministic comparison, and evidence-aware communication.

The most important engineering lesson is the fairness protocol. Without common graph inputs, common initial conditions, common budget, common duration, and stable event-level pseudo-random draws, a comparative dashboard can generate visually convincing but scientifically weak claims. By treating these fields as explicit metadata, the project makes the experimental conditions inspectable. The second lesson is the value of boundaries: OSM road geometry is useful but does not make an epidemic forecast; an LLM explanation can aid comprehension but does not decide; a 3D miniature can increase engagement but must retain topology; a portable executable is stronger when its core demonstration does not depend on a public map service.

For assessment, the recommended presentation sequence is: reset to the standard teaching scenario; explain the graph and budget; demonstrate a fair comparison; inspect the resulting action plan; add symptom evidence to show the Bayesian update; use a manual what-if closure;

vary one parameter in sensitivity analysis; then step through the five curated scenarios. The presenter should close by stating that each numerical output is a synthetic projection under stated assumptions. That statement is not a weakness. It is the condition that lets the mathematical content remain honest, reproducible, and teachable.

## 15 Decision register: implementation choices and rationale

This register turns the development checklist into an auditable technical review. It records the material choices that shaped the delivered system, their motivation, and the trade-off accepted. The entries are grouped by dependency rather than chronological order; a later entry may refine an earlier one. Generated template boilerplate is excluded because it was not a project decision.

## 15.1 A. Mathematical domain and graph foundations

| ID  | Decision                                                      | Why it was made and trade-off                                                                                                                                                      |
|-----|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| D01 | Use a city graph rather than a raster map                     | Graph vertices and edges make degree, shortest paths, centrality, and cuts directly computable. A conventional map layer would add visual realism but hide the discrete structure. |
| D02 | Separate population locations from road connectors            | Junctions preserve route continuity but do not invent a population compartment. The trade-off is a simplified connector traversal rather than a full traffic model.                |
| D03 | Provide homes, schools, hospitals, offices, blocks, and roads | Facility types make the city legible and support population heterogeneity. They remain scenario labels rather than observed land-use or occupancy data.                            |
| D04 | Make synthetic generation seed-driven                         | A fixed seed makes a graph and outcome repeatable for teaching, tests, and reports. It sacrifices the apparent novelty of a new random map on every refresh.                       |
| D05 | Guarantee connected structure by construction                 | Algorithms should have meaningful paths to compare; disconnected random graphs often create arbitrary no-path outcomes.                                                            |
| D06 | Use road classes, distances, probabilities, and capacities    | One edge record can support visual width, probability routes, and min-cut structure. Values are proxies, not measurements.                                                         |
| D07 | Use bounded graph-size controls                               | Limits protect browser rendering and algorithm responsiveness. The trade-off is that citywide simulation is deliberately unsupported.                                              |
| D08 | Preserve stable graph/edge/node identifiers                   | Actions, reports, tests, and fixtures must refer to the same objects across components.                                                                                            |
| D09 | Use Haversine distance for imported geometry                  | It gives a standard geographic crop/length calculation without needing a map SDK. It does not account for routing curvature beyond supplied way points.                            |
| D10 | Use imported-junction population proxy                        | The common engine needs $N_v > 0$ at populated imported junctions. This allows a lesson but must never be read as census data.                                                     |

## 15.2 B. Epidemic engine and probability choices

| ID  | Decision                                                        | Why it was made and trade-off                                                                                                                             |
|-----|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| D11 | Use a discrete daily SIR-style state update                     | It is intelligible to a second-year audience and supports animation. It is less expressive than continuous or agent-based epidemiology.                   |
| D12 | Track deceased separately from recovered                        | The brief asked for mortality outcomes, and a separate $D$ count makes transition order and reporting explicit.                                           |
| D13 | Use binomial event draws                                        | Individual opportunity logic gives $E[Y] = np$ and bounded counts. It is a simplified independence assumption.                                            |
| D14 | Recover before modeled mortality each day                       | This fixes an unambiguous transition order and prevents a person being both recovered and deceased in the same step.                                      |
| D15 | Combine exposures with a no-event complement                    | It avoids invalid probability sums and treats multiple sources transparently. It assumes independent opportunities.                                       |
| D16 | Multiply road opportunity probabilities along a connector route | Sequential travel suggests a product and preserves road-chain influence. It omits detailed route choice and time.                                         |
| D17 | Cap per-contact probability at 0.95                             | The input remains valid and avoids extreme values. The cap is an implementation safeguard, not a clinical fact.                                           |
| D18 | Use keyed-hash pseudo-random trials                             | Each node/day/event draw is stable under comparison, avoiding sequential RNG drift. It is deterministic pseudo-randomness rather than a physical process. |
| D19 | Use an initial-infection fallback                               | A blank selection still creates a teachable run. The fallback is deterministic and visible, avoiding silent all-zero results.                             |
| D20 | Label every output a scenario projection                        | The model is not calibrated, so forecasts or clinical claims would be misleading.                                                                         |

## 15.3 C. Intervention and comparison choices

| ID  | Decision                                                  | Why it was made and trade-off                                                                                                                                                    |
|-----|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| D21 | Compare five established strategies                       | The course objective is understanding/comparison, not invention. The selected set spans baseline, local connectivity, bridge connectivity, route likelihood, and cut separation. |
| D22 | Include seeded Random                                     | A baseline prevents overclaiming and supports a chance-alignment teaching case. It is not framed as recommended policy.                                                          |
| D23 | Use degree as a local-hub heuristic                       | It is simple to explain and test. It may miss low-degree structural bridges.                                                                                                     |
| D24 | Use unweighted Brandes betweenness                        | Topological bridges are the lesson and the runtime is bounded. It does not use probability-weighted centrality.                                                                  |
| D25 | Transform route probability by $-\ln(\tau)$               | Products become additive nonnegative costs so Dijkstra applies. It assumes an edge probability can stand in for route opportunity.                                               |
| D26 | Target five largest non-infected destinations in Dijkstra | This creates a small meaningful destination set and remains fast. It is a design heuristic rather than a complete all-pairs calculation.                                         |
| D27 | Run min-cut from source to hospital/large destination     | It gives an interpretable containment story. The target selection is a policy proxy, not a full multi-sink flow optimization.                                                    |
| D28 | Use road-class capacity proxy                             | Required for min-cut and visible in the graph model. It is not observed movement capacity.                                                                                       |
| D29 | Constrain roads, nodes, and population                    | A budget becomes enforceable and comparable. Monetary/social costs are not fabricated.                                                                                           |
| D30 | Select feasible nodes before roads                        | The selection procedure is deterministic and readable. It is not a joint global optimum.                                                                                         |
| D31 | Use lower infections then modeled deaths as winner order  | This makes the scorecard decisive while revealing remaining metrics. It is not a normative real policy function.                                                                 |
| D32 | Represent equal outcomes as ties                          | Prevents false unique winners created by sort order and improves scientific honesty.                                                                                             |
| D33 | Keep manual actions outside algorithm comparison          | Exploration should not mutate or contaminate a fair five-way result.                                                                                                             |

## 15.4 D. Bayesian, recommendation, and experiment choices

| ID  | Decision                                                         | Why it was made and trade-off                                                                                     |
|-----|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| D34 | Use explicit Bayes-rule hotspot update                           | It demonstrates prior, likelihood, posterior, and evidence strength visibly. It is not a diagnostic model.        |
| D35 | Build prior from exposure plus current simulation probability    | It connects graph dynamics to evidence reasoning. Coefficients are illustrative rather than estimated.            |
| D36 | Let users enter symptom count, observed population, and strength | Inputs make the update inspectable and support “what if” teaching. They are constrained to valid ranges.          |
| D37 | Write recommendations from winning actions                       | A score becomes a concrete target/rationale/cost line. The recommendation text remains conditional.               |
| D38 | Keep LLM explanation optional and downstream                     | Plain language helps decision-makers, but numerical calculations must remain deterministic and inspectable.       |
| D39 | Use a compact LLM payload                                        | It avoids server input-limit failure and prevents unnecessary graph serialization.                                |
| D40 | Record fairness metadata in the comparison                       | Users and exported reports can audit which conditions were held fixed.                                            |
| D41 | Provide sensitivity sweep rather than confidence labels          | Directional parameter dependence can be taught honestly; calibrated uncertainty cannot be claimed.                |
| D42 | Create five curated strategy-winning scenes                      | A presentation needs distinct examples. Each scene teaches contextual performance rather than global superiority. |
| D43 | Make demonstration order fixed                                   | A presenter can reliably step through the same proof-of-concept sequence.                                         |

## 15.5 E. Interface, visualisation, and teaching choices

| ID  | Decision                                                         | Why it was made and trade-off                                                                                                          |
|-----|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| D44 | Use a desktop-first five-step ribbon workspace                   | The target is a laptop classroom demonstration; steps reduce control overload. Mobile-specific layouts were deliberately out of scope. |
| D45 | Show timeline playback and scrubbing                             | The dynamic model becomes inspectable at every day, not only at a final chart.                                                         |
| D46 | Provide 2D graph zoom, pan, selection, reset                     | Learners need direct structural inspection and recovery from a lost viewport.                                                          |
| D47 | Provide 3D architectural miniature                               | Visual engagement improves while preserving visible topology. It avoids false photorealism.                                            |
| D48 | Add facility geometry, road markings, parcels, buffers, lighting | These visual cues make districts and intervention points readable without altering graph semantics.                                    |
| D49 | Display outbreak halos and quarantine fences                     | The intervention and state change remain visible in 3D rather than only in a table.                                                    |
| D50 | Use global light/dark tokens                                     | Theme consistency avoids page-by-page contrast errors and persists user preference.                                                    |
| D51 | Honor reduced-motion preferences                                 | Animation is supportive, not required for comprehension.                                                                               |
| D52 | Offer standard scenario reset                                    | Presenter recovery must be one-click; it also protects repeatable demonstrations.                                                      |
| D53 | Add presenter notes/script                                       | Mathematical interpretation needs a concise spoken path, not only controls.                                                            |
| D54 | Add CSV, PDF, and share page                                     | A project needs evidence beyond live UI. Multiple exports serve data, printable, and link-based review.                                |

## 15.6 F. External-data, resilience, and desktop choices

| ID  | Decision                                                | Why it was made and trade-off                                                                                             |
|-----|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| D55 | Use bounded OSM imports                                 | Real road geometry is engaging, but citywide import would be fragile, slow, and too complex for the lesson.               |
| D56 | Use Overpass then official XML fallback                 | Different public routes reduce availability risk. The system still communicates failure rather than fabricating geometry. |
| D57 | Cache imports for 15 minutes                            | This respects public infrastructure and improves repeated classroom use.                                                  |
| D58 | Clip ways before node limit                             | The cap measures the selected neighborhood, not unrelated geometry continuing beyond it.                                  |
| D59 | Surface oversized/empty/unavailable errors              | Failure modes are teachable and recoverable instead of indefinite loading.                                                |
| D60 | Keep OSM attribution                                    | Geometry source must remain clear in graph/report data.                                                                   |
| D61 | Bundle fixture JSON for demos                           | A classroom demo must not depend on upstream OSM availability or changing data.                                           |
| D62 | Use Electron portable Windows x64                       | The user preferred a robust desktop artifact; the shell reuses the full-stack codebase.                                   |
| D63 | Start local Express loopback in Electron                | Server procedures work locally without hosted dependencies.                                                               |
| D64 | Use local desktop JSON persistence                      | Core desktop saves should work without database access.                                                                   |
| D65 | Exclude Vite development plugin from production runtime | A packaged desktop app cannot rely on build-only JSX tooling.                                                             |
| D66 | Validate package contents and checksum                  | Runtime inspection catches packaging errors that source tests alone cannot.                                               |

## 15.7 G. Verification, release, and handoff choices

| ID  | Decision                                           | Why it was made and trade-off                                                                                                             |
|-----|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| D67 | Write pure-module tests for math and visuals       | Logic remains verifiable without browser/3D runtime noise.                                                                                |
| D68 | Add rendered interaction tests                     | A control that is present but fails to update state is a product defect, even if pure functions pass.                                     |
| D69 | Document QA matrix and validation summary          | Release evidence should be reviewable and not depend on oral memory.                                                                      |
| D70 | Keep live OSM verifier opt-in                      | Third-party availability should not make the deterministic local release suite flaky.                                                     |
| D71 | Check TypeScript at release gate                   | Shared contracts and component refactors can introduce unsafe mismatches caught before delivery.                                          |
| D72 | Run full 62-test release suite                     | Regression coverage protects earlier completed capabilities during finishing work.                                                        |
| D73 | Save project checkpoints                           | The user can restore auditable milestone states rather than trusting a mutable working directory.                                         |
| D74 | Create a private GitHub repository                 | Source control and project handoff are preserved without exposing work by default.                                                        |
| D75 | Provide Windows CLI guidance                       | The user completed their own authenticated repository push and learns a reproducible workflow.                                            |
| D76 | Attach final portable executable separately        | The requested primary deliverable is usable without a hosted site.                                                                        |
| D77 | Leave external Windows smoke test honestly pending | A Linux sandbox cannot prove every Windows device will launch; recording that uncertainty is more rigorous than pretending it was tested. |

## 16 Appendices

## 16.1 Appendix A — Worked numerical day update

Take one source  $u$  with  $I_u = 6$ , road probability  $\tau_e = 0.04$ , and multiplier  $\alpha = 0.20$ . The following computation is the exact numerical interpretation of the daily edge-opportunity logic.

$$\begin{aligned} x_e &= \alpha \tau_e = 0.20 \times 0.04 = 0.008 \\ q_e &= 1 - (1 - x_e)^{I_u} = 1 - 0.992^6 \approx 0.0470 \\ p_v &= 1 - (1 - 0.0470)(1 - 0.030) = 0.0756 \end{aligned}$$

Now set  $S_v = 120$ ,  $I_v = 10$ ,  $\rho = 0.06$ , and  $\mu = 0.006$ . The expected transition counts (not the actual deterministic draws) are

$$\begin{aligned} E[R_v^+] &= 10(0.06) = 0.6000 \\ E[D_v^+ \text{ after recovery}] &\approx (10 - 0.6000)(0.006) = 0.0564 \\ E[I_v^+] &= 120(0.0756) = 9.0720 \end{aligned}$$

If the keyed trials produce  $R_v^+ = 1$ ,  $D_v^+ = 0$ , and  $I_v^+ = 8$ , the count state moves from  $(S_v, I_v, R_v, D_v) = (120, 10, 0, 0)$  to  $(112, 17, 1, 0)$ . The total remains 130. The distinction between these expectations and a reproducible integer trial is essential: the former summarizes repeated draws, while the latter is the scenario trace seen in the timeline.

## 16.2 Appendix B — Worked Bayesian update

Assume modeled probability  $p_v = 0.08$ , exposure  $h_v = 0.10$ ,  $a = 18$  symptomatic observations,  $n = 120$  observations, and evidence strength  $k = 0.80$ . The illustrative update becomes

$$P(H) = 0.015 + 0.55(0.08) + 0.65(0.10) = 0.1240$$

$$z = 18 / 120 = 0.1500$$

$$L = 0.18 + 0.65(0.15)(0.80) = 0.2580$$

$$F = 0.04 + 0.12(0.15)(0.80) = 0.0544$$

$$P(H | E) = [0.2580 \times 0.1240] / [0.2580 \times 0.1240 + 0.0544 \times 0.8760] \approx 0.4010$$

The hotspot score therefore rises from 12.4% to about 40.1% under the stated likelihood assumptions. It is an algebraic teaching calculation, not a diagnostic conclusion.

## 16.3 Appendix C — Complexity and algorithmic scale

| Operation                         | Illustrative complexity                            | Why bounded scale matters                                                               |
|-----------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------|
| Synthetic graph build             | $O( V  +  E )$                                     | Block/ring construction is linear in created objects.                                   |
| One SIR day, simplified traversal | Depends on active sources and traversed connectors | Small graphs make the transparent connector queue practical.                            |
| Degree scores                     | $O( V  +  E )$                                     | Every edge increments two endpoints.                                                    |
| Brandes betweenness, unweighted   | $O( V  E )$                                        | A cap prevents centrality from dominating a desktop interaction.                        |
| Dijkstra, simple unvisited scan   | $O( V ^2 +  E )$ per source/target                 | Only initial sources and five major destinations are sampled.                           |
| Augmenting-path max flow          | Input dependent                                    | The graph cap and a single prioritized target keep the teaching computation responsive. |
| Export/report creation            | Linear in serialized scenario size                 | Large maps are rejected before they become large report payloads.                       |

## 16.4 Appendix D — Exact interface contract summary

| Contract            | Fields and semantic role                                                                                            |
|---------------------|---------------------------------------------------------------------------------------------------------------------|
| CityNode            | Identifier, label, kind, facility type, population, block/district, planar position, optional OSM ID/coordinates.   |
| CityEdge            | Identifier, source/target IDs, distance, transmission probability, capacity, road class, optional label/OSM way ID. |
| EpidemicParameters  | Transmission rate $\alpha$ , recovery $\rho$ , mortality $\mu$ , days $T$ , initial infected IDs, seed $\sigma$ .   |
| InterventionBudget  | Maximum road closures $b_e$ , maximum quarantined nodes $b_v$ , maximum quarantined population $b_p$ .              |
| InterventionAction  | Road closure, node quarantine, or block isolation, including target and human-readable reason.                      |
| Node snapshot       | State label, infection probability, hotspot score, and four count compartments.                                     |
| StrategyOutcome     | Actions, budget use, timeline, hotspots, final/peak metrics, duration, containment, components, rationale.          |
| Fairness metadata   | Graph ID/fingerprint, initial infection IDs, budget, full epidemic parameters, seed, deterministic trial method.    |
| Scenario comparison | Graph, evidence, five outcomes, winner, tie state, leading strategies, timestamp, disclaimer.                       |

## 16.5 Appendix E — Test and review matrix

| Area                  | Representative positive, boundary, and recovery checks                                                     |
|-----------------------|------------------------------------------------------------------------------------------------------------|
| Graph builder         | Seed reproducibility; connected graph; facility types; no invalid population/edge fields.                  |
| Engine                | Zero transmission; recoveries/deaths; quarantined node risk; valid probability bounds; count conservation. |
| Strategies            | Budget respected; deterministic baseline; expected candidate ranking; tie-aware result.                    |
| Bayes/recommendations | No-evidence explanation; positive evidence rank; bounded posterior; target/cost/rationale text.            |
| OSM importer          | Normal fixture parse; invalid coordinate/radius; node cap; timeout/error; fallback/cache paths.            |
| Workspace controls    | Minimum/maximum input clamping; mode switch; importer pending/error; manual actions.                       |
| Playback/analysis     | Day bounds; play/pause; sensitivity parameter change; reset to standard scenario.                          |
| Graph UI              | Zoom; pan; selection; reset viewport; capped initial selections.                                           |
| Theme                 | Light/dark update; persistence; document-class cleanup.                                                    |
| Export/persistence    | CSV action; PDF component error behavior; scenario save/share contract.                                    |
| Desktop               | Runtime staging; no production Vite reference; fixture content in packaged assets; local store behavior.   |

## 16.6 Appendix F — Bibliography and implementation evidence

- [1] W. O. Kermack and A. G. McKendrick, “A Contribution to the Mathematical Theory of Epidemics,” **Proceedings of the Royal Society A**, 1927. [doi:10.1098/rspa.1927.0118](https://doi.org/10.1098/rspa.1927.0118)
- [2] J. A. P. Heesterbeek, “The Kermack–McKendrick epidemic model revisited,” **Mathematical Biosciences**, 2005. [PubMed record](#)
- [3] M. J. Keeling and K. T. D. Eames, “Networks and epidemic models,” **Journal of the Royal Society Interface**, 2005. [doi:10.1098/rsif.2005.0051](https://doi.org/10.1098/rsif.2005.0051)
- [4] U. Brandes, “A Faster Algorithm for Betweenness Centrality,” **Journal of Mathematical Sociology**, 2001. [doi:10.1080/0022250X.2001.9990249](https://doi.org/10.1080/0022250X.2001.9990249)
- [5] L. R. Ford Jr. and D. R. Fulkerson, “A Simple Algorithm for Finding Maximal Network Flows and an Application to the Hitchcock Problem,” **Canadian Journal of Mathematics**, 1956. [doi:10.4153/CJM-1956-045-5](https://doi.org/10.4153/CJM-1956-045-5)
- [6] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” **Numerische Mathematik**, 1959. [doi:10.1007/BF01386390](https://doi.org/10.1007/BF01386390)
- [7] C. M. Bishop, **Pattern Recognition and Machine Learning**, Bayesian inference foundations. [Springer book page](#)
- [8] OpenStreetMap, “Copyright and License.” [openstreetmap.org/copyright](https://openstreetmap.org/copyright)
- [9] Overpass API documentation, “Preface.” [Overpass documentation](#)
- [10] OpenStreetMap Wiki, “API v0.6: Map data by bounding box.” [OSM Wiki](#)

Project implementation evidence consulted for this report: shared contracts in `shared/epidemic.ts`; engine in `shared/epidemicEngine.ts`; strategies in `shared/interventions.ts`; preset catalog in `shared/demoPresets.ts`; importer in `server/osmImport.ts`; desktop package in `desktop/main.cjs` and `scripts/prepare-desktop-runtime.mjs`; quality evidence in `docs/qa-validation-summary.md` and `docs/qa-test-matrix.md`; packaging boundaries in `docs/windows-portable-package.md`; and the project checklist `todo.md`.

## 17 Appendix E — Formal model, state-machine, and data-contract audit

This appendix is deliberately more formal than the interface documentation. It answers a different question: **what object is the program actually computing?** A user sees a city, a timeline, and five recommendations. The engine instead consumes a finite graph, a parameter vector, optional interventions, a deterministic key stream, and a horizon; it returns a finite sequence of count states and summary statistics. Treating these as separate layers was a major design decision. It prevents a visually persuasive map from silently becoming the mathematical model, and it makes each calculation inspectable in a second-year discrete mathematics setting.

### 17.1 E.1 Domains, notation, and typing discipline

Let the scenario graph be  $G = (V, E)$ . Every vertex has a stable string identifier; every edge has a stable identifier and two endpoint identifiers. The graph is simple at the teaching-model level even when imported geometry originally contains a more complicated road representation. A vertex belongs to one semantic category: home, school, hospital, office, or road intersection. The first four categories can carry a nonnegative modeled population; the last category is an explicitly non-population-bearing routing connector.

```
V = V_home u V_school u V_hospital u V_office u V_road
V_i n V_j = ∅ for i ≠ j
N_v ∈ {0,1,2,...}; N_v = 0 for v ∈ V_road
E ⊆ {{u,v} : u,v ∈ V, u ≠ v}
τ_e ∈ [0,1]; d_e > 0; class_e ∈ {primary, secondary, local}
```

The application uses a **typed graph** rather than a generic adjacency matrix because semantics matter. A school can be suggested for quarantine; an ordinary road-intersection cannot consume a population-quarantine budget. A hospital can receive special priority as a max-flow destination; a home should not. This distinction is not a claim that real facilities have homogeneous risk. It is a model vocabulary chosen to make the intervention constraints legible.

The epidemic state associated with a population-bearing vertex  $v$  on day  $t$  is a four-tuple  $X_{v(t)} = (S_{v(t)}, I_{v(t)}, R_{v(t)}, D_{v(t)})$ . The interface also uses a quarantined display state. Quarantine is an **intervention flag**, not a fifth biological compartment: it changes available connectors or blocks movement but does not redefine a person’s biological status. This avoids the common modelling ambiguity in which “quarantine” is incorrectly counted both as a disease state and as a policy action.

```
S_v(t), I_v(t), R_v(t), D_v(t) ∈ {0,1,2,...}
S_v(t)+I_v(t)+R_v(t)+D_v(t) = N_v
I_v(t) = 0 and S_v(t) = 0 for v ∈ V_road
```

The equality is an invariant, not an optional diagnostic. After every transition, the implementation bounds each sampled count against the people available for that transition, then computes the next susceptible count by conservation. A useful audit assertion is therefore

```
for every v and every completed day t:
  0 ≤ S_v(t), I_v(t), R_v(t), D_v(t) ≤ N_v
  S_v(t)+I_v(t)+R_v(t)+D_v(t) = N_v
```

This exact check matters more than an attractive epidemic curve. A curve can look plausible while violating person conservation through an off-by-one recovery count. The project's data contract and tests make the invariant explicit.

## 17.2 E.2 Parameter vector and admissible ranges

The scenario parameter vector is

$$\theta = (\alpha, \rho, \mu, T, I_0, \sigma, B_e, B_v, B_p)$$

where  $\alpha$  is the global transmission multiplier,  $\rho$  is recovery probability,  $\mu$  is mortality probability,  $T$  is the number of simulated days,  $I_0$  is the selected initial infection count or initial source set,  $\sigma$  is the deterministic seed, and the final three quantities are the road, node, and population budgets. The notation intentionally separates  $\tau_e$ , the road-level base opportunity, from  $\alpha$ , the scenario-wide multiplier. A map can retain its geometry while students examine what happens when the common multiplier changes.

Every editable control is clamped at its boundary rather than accepted as an arbitrary JavaScript number. This is both an interface and a modelling decision. Negative transmission, a recovery probability above one, a negative road budget, and a nonintegral horizon do not represent additional experimental cases; they represent malformed inputs. Boundary clamping gives a stable educational response and keeps the model's codomain meaningful.

| Quantity        | Admissible domain                 | Why the boundary exists                          | Consequence of invalid input           |
|-----------------|-----------------------------------|--------------------------------------------------|----------------------------------------|
| $\alpha$        | nonnegative bounded control range | Prevents a negative probability scale            | Clamped to nearest permitted value     |
| $\rho, \mu$     | $[0, 1]$                          | Required for Bernoulli transition interpretation | Clamped to 0 or 1                      |
| $T$             | positive integer range            | Timeline needs a finite discrete index           | Rounded and bounded                    |
| $B_e, B_v, B_p$ | nonnegative integers              | A budget cannot buy negative actions             | Clamped to zero or upper control limit |
| $\sigma$        | finite integer                    | Reproducibility key must be serializable         | Normalized to an integer               |

The engine treats  $\rho$  and  $\mu$  sequentially, with recovery sampled before mortality among the remaining infected count. The ordering is documented because two superficially similar formulas are different. If mortality were sampled from the original infected pool, one individual could appear in both recovery and mortality draws. Sequential sampling instead encodes mutually exclusive outcomes within one day: recover, then die from those who remain infected, then remain infected. This is a bookkeeping rule, not a fitted clinical process.

## 17.3 E.3 Intervention transformation as a pure graph operation

Let  $A = (E_A, V_A)$  denote selected road closures and selected quarantined locations. The intervention planner produces  $A$ ; the epidemic engine receives an active graph derived from  $A$ . Separating planning from execution was essential for fair comparison. It means the engine never needs to know whether an edge was selected by Random, Highest Degree, Betweenness, Dijkstra, Max-Flow/Min-Cut, or a manual user choice.

$$\begin{aligned} E_{\text{active}}(A) &= \{ e \in E : e \notin E_A \text{ and neither endpoint is quarantined by } V_A \} \\ V_{\text{active}}(A) &= V \setminus V_A \text{ for movement and exposure purposes} \end{aligned}$$

The implementation's exact representation may retain a quarantined vertex for display, but it removes its usable connectors for the transmission traversal. This distinction preserves explainability: the map can still show **which location was quarantined** while the graph calculation correctly treats it as disconnected. A closure is not silently removed from the visual graph, and a display-only fence is not allowed to leave the edge mathematically active.

Budget feasibility is lexicographic rather than an implicit optimization solver. Candidates arrive in a strategy-specific order. Nodes are considered first; any candidate whose modeled population would exceed  $B_p$  is skipped; selected nodes are limited by  $B_v$ . Roads are subsequently selected in candidate order until  $B_e$  is reached. Formally, each reported plan satisfies

$$\begin{aligned} |E_A| &\leq B_e \\ |V_A| &\leq B_v \\ \sum_{(v \in V_A)} N_v &\leq B_p \end{aligned}$$

This produces a reproducible feasible plan. It does **not** claim to solve a global knapsack or mixed integer program jointly over nodes and edges. That non-claim is important. A globally optimized containment design would be a different research project with different assumptions and would obscure the intended comparison among established graph heuristics.

## 17.4 E.4 Force of infection and daily update order

For a traversable opportunity from infected source  $u$  along edge  $e = \{u, v\}$ , define a capped elementary probability  $x_e = \text{clip}(\alpha \tau_e, 0, 0.9999)$ . If  $I_{u(t)}$  infected individuals are used as repeated simplified opportunities, the edge-level daily probability is

$$q_{(u \rightarrow v, e)}(t) = 1 - [1 - x_e]^{(I_u(t))}$$

The exponent is not a population-contact survey. It is a transparent independence approximation: the probability that no one of  $I_{u(t)}$  simplified opportunities succeeds is the product of their no-event probabilities; the complement is at least one success. The cap below one avoids a singular transformed Dijkstra cost and avoids numeric degeneracy in a finite-precision interface.

If several eligible sources reach the same destination, the engine aggregates their probability contributions by the complement of all sources failing. For source opportunity set  $U_{v(t)}$ ,

$$p_v(t) = 1 - \prod_{u \in U_v(t)} [1 - q_{(u \rightarrow v)}(t)]$$

This is a standard union-of-independent-opportunities approximation. It has two useful teaching properties. First, it is bounded in  $[0, 1]$  without ad hoc rescaling. Second, it visibly shows diminishing increments: a second route raises risk, but not by naively adding probabilities beyond one. It is also limited: routes and people are not independent in a real city, so the product is an educational simplification.

The day transition uses a frozen state. All day- $t + 1$  quantities are derived from day  $t$  counts and the day- $t$  active graph, then committed together. The program does not update the first vertex and immediately use its new infection count when updating the next vertex. That asynchronous alternative can create an unintended vertex-order effect. The synchronous rule is

```
new infections:  I_v+ ~ Binomial(S_v(t), p_v(t))
recoveries:      R_v+ ~ Binomial(I_v(t), ρ)
deaths:          D_v+ ~ Binomial(I_v(t) - R_v+, μ)
S_v(t+1) = S_v(t) - I_v+
I_v(t+1) = I_v(t) + I_v+ - R_v+ - D_v+
R_v(t+1) = R_v(t) + R_v+
D_v(t+1) = D_v(t) + D_v+
```

The negative terms in the infected update appear only after the new infections have been added, and recoveries/deaths are bounded by the original infected count. With the availability checks described earlier, the conservation invariant follows directly by summing the four update equations. This is the report's central mathematical justification for calling the engine a discrete-time SIR-with-deaths scenario model.

## 17.5 E.5 Deterministic pseudo-random trials

The project wanted both stochastic-looking epidemic transitions and fair, repeatable strategy comparison. The solution is deterministic keyed Bernoulli sampling. Each conceptual draw is assigned a key derived from scenario seed, day, vertex or edge identity, transition type, and trial index. A stable hash turns the key into a normalized number in  $[0, 1)$ . A trial succeeds precisely when that number is below the requested probability.

```
U = normalizedHash(σ, day, entityId, transitionKind, trialIndex)
Bernoulli(p) = 1 if U < p, otherwise 0
Binomial(n,p) = Σ_{j=0}^{n-1} Bernoulli_j(p)
```

This is not cryptographic randomness, nor does it claim a statistically ideal pseudo-random generator. It is a reproducibility device. The same inputs produce the same daily trace; changing

an intervention changes only the parts of the graph or state that logically change. The design supports common random numbers: plans are evaluated against the same seed and key scheme, reducing accidental variability in their differences. A later research extension could evaluate many independent seeds and report a distribution; the present application correctly labels its outputs as scenario projections rather than confidence intervals.

## 17.6 E.6 Derived metrics and semantic boundaries

The dashboard exposes cumulative infections, active infections, modeled deaths, peak active infections, peak day, duration, and containment fraction. Each measure is intentionally described as a model output. For example, cumulative infection is counted from the scenario's initial infections plus simulated new infections; it is not a case-reporting estimate. Modeled deaths are a count generated by the supplied mortality transition; they are not a mortality forecast. Peak day is an argmax over the finite timeline, with deterministic tie handling according to the engine's traversal order.

Containment is relative to the corresponding no-intervention baseline. If  $C_0$  is baseline cumulative infections and  $C_A$  is the cumulative infections under plan  $A$ ,

$$\begin{aligned}\eta(A) &= \text{clip}_{(0,1)} \left[ (C_0 - C_A) / C_0 \right] \text{ for } C_0 > 0 \\ \eta(A) &= 0 \text{ otherwise}\end{aligned}$$

The zero-baseline branch avoids division by zero and says something semantically useful: if no baseline infections occur, no plan can claim an infection reduction. The explicit clamp avoids an unstable negative percentage being presented as a percent beyond the intended scale. Importantly, the system's primary winner rule does not maximize  $\eta$  directly; it compares lower final cumulative infection first and modeled mortality second. Containment remains an explanatory secondary metric rather than a hidden objective that could distort an intuitive result.

The formal audit yields one final conclusion. EpiGraph is neither a continuous ODE model nor a real-time public-health surveillance system. It is a finite, typed, graph-constrained, discrete-time stochastic scenario machine with deterministic replay. Every visual and recommendation layer should be interpreted within that boundary.

## 18 Appendix F — Algorithm execution trace and strategy-by-strategy audit

The simulator deliberately compares five established methods rather than inventing a black-box “best intervention” algorithm. That choice keeps the project appropriate for a second-year discrete mathematics course: each method has a recognisable graph-theoretic idea, a visible failure mode, and a different structural hypothesis. This appendix records the operational trace of every strategy. The decisive distinction is between candidate generation and feasible-plan construction. A strategy may rank every vertex and edge; the common budget filter decides which ranked candidates become actual closures or quarantines.

### 18.1 F.1 Shared comparison skeleton

All strategies enter the same runner. The runner freezes the scenario graph, parameters, evidence, horizon, initial infections, budgets, and deterministic seed before candidate generation begins. It calculates a no-algorithm baseline once, then evaluates one plan per strategy. The comparison uses a fresh copy of the graph and fresh state for each plan. No strategy inherits a road closure, recovery, infection, or random trial from the strategy evaluated immediately before it.

```
input: graph G, parameter vector  $\theta$ , evidence E, budget B, initial state  $X(0)$ , seed  $\sigma$ 

baseline = simulate(G,  $\theta$ , E, no actions,  $X(0)$ ,  $\sigma$ )

for h in {Random, Degree, Betweenness, Dijkstra, MinCut}:
    candidates_h = rankCandidates(G,  $\theta$ , E, h)
    A_h = commonBudgetFilter(candidates_h, B)
    result_h = simulate(G,  $\theta$ , E, A_h,  $X(0)$ ,  $\sigma$ )

compare results by (final cumulative infections, modeled deaths)
mark tied if the best metric tuple occurs more than once
```

This skeleton was chosen instead of letting every strategy decide its own budget semantics. Without a common filter, a strategy that recommends more closures could look better merely because it spent more resource. It was also chosen instead of reseeding every run. Different seeds would make a performance difference ambiguous: was the intervention better, or did it receive a luckier random trace? Common keyed trials do not make a model objectively true, but they make within-scenario comparison auditable.

### 18.2 F.2 Random: required baseline, not an endorsement

Random takes the eligible population-bearing nodes and roads, applies a deterministic seeded shuffle, and passes the shuffled lists to the common budget filter. The seed offset is fixed relative to

the scenario seed, so the same scenario always gives the same random plan. The result is therefore **reproducibly random** in the educational sense.

```
eligibleNodes = populationBearingVertices(G)
eligibleEdges = roads(G)
nodeOrder = seededShuffle(eligibleNodes, σ + constant_1)
edgeOrder = seededShuffle(eligibleEdges, σ + constant_2)
return { nodes: nodeOrder, edges: edgeOrder }
```

The Random strategy is included to establish a null baseline: it shows what a budget allocation looks like when it ignores graph structure. It is not included to imply that chance is a public-health policy. The system explicitly guards against a communicative error that would be especially awkward in a demonstration: if Random happens to tie or appear first in a non-tie sort, the interface must not describe it as a unique superior method. The tie-aware scorecard reports an inconclusive result when the best tuple is shared.

Random can occasionally produce a unique win in a deliberately curated teaching fixture. The correct interpretation is narrow. A randomly chosen road can happen to coincide with a critical bridge or high-probability route. That event demonstrates why one scenario is not a universal method ranking. It does not overturn the structural motivation for the other four algorithms.

### 18.3 F.3 Highest Degree: local connectivity heuristic

The degree of a vertex is the number of incident edges. For undirected graph  $G$ , the implementation counts every road at both endpoints. A vertex candidate is ranked by decreasing degree, with stable identifier order resolving equal scores. An edge candidate is ranked by the sum of its endpoint degrees. This edge score is not a separate theorem; it is an interpretable induced heuristic that favors roads connected to locally well-connected places.

```
d(v) = | { e ∈ E : v is an endpoint of e } |
nodeScore(v) = d(v)
edgeScore({u,v}) = d(u) + d(v)
```

The method's hypothesis is local: contacts and potential routes concentrate around highly connected places. If a graph has a school, office, or junction at the center of many roads, placing a constrained intervention there can remove several immediately available connectors. This method is cheap to compute—one pass over all edges produces every degree—and easy to explain at the interface.

It can nevertheless fail. Two dense districts may each contain high-degree vertices, while a low-degree connector lies between them. Closing the connector can separate the districts more effectively than closing a local hub. It can also be confounded by imported geometric detail: an area with many short map segments can have high topological degree without representing

proportionally greater meaningful movement. These limitations justify comparing degree with betweenness and cut-based methods instead of labeling it “optimal.”

## 18.4 F.4 Betweenness: shortest-path bridge heuristic

Betweenness centrality asks how frequently a vertex occurs on shortest paths between other vertices. Let  $\sigma_{s,t}$  be the number of shortest paths from source  $s$  to target  $t$ , and let  $\sigma_{s,t}(v)$  be the number that pass through  $v$ . The conceptual score is

$$C_B(v) = \sum_{(s \neq v \neq t)} \sigma_{s,t}(v) / \sigma_{s,t}$$

The code follows the unweighted Brandes-style structure. For each source, it performs breadth-first discovery, records distances, predecessor lists, and counts of shortest paths, then processes the discovery stack in reverse to accumulate dependencies. Because the study graph is undirected, the final score divides paired ordered-source contributions appropriately. Candidate edges receive the sum of their endpoint scores, providing a consistent node-and-road selection interface.

```
for source s:
  BFS from s
  for each reached w: record predecessors P[w] and path count  $\sigma[w]$ 
  process stack in reverse BFS order
  pass dependency  $\delta[w]$  backward through P[w]
  add  $\delta[w]$  to the centrality score of non-source vertices
```

The method’s hypothesis is global but topological: a high-score vertex lies on many **shortest** connector sequences. It does not know actual travel time, traffic volume, social mixing, or capacity unless these coincide with the graph topology. This distinction belongs in the report because “central” in graph theory is not identical to “important” in epidemiology. The algorithm becomes pedagogically valuable precisely because the learner can contrast its bridge intuition with the local-hub intuition of degree.

At typical capped teaching sizes, the unweighted procedure is responsive enough for interaction. A more exact weighted centrality calculation could be included, but it would complicate the explanation and would blur the contrast with Dijkstra, which already demonstrates probability-weighted paths. The selected implementation therefore trades physical realism for conceptual separability.

## 18.5 F.5 Dijkstra route blocking: probability as additive cost

Dijkstra’s algorithm requires nonnegative additive edge weights. Epidemic transmission along a route is naturally presented as a product of simplified per-edge probabilities. The project uses a negative logarithm transformation to bridge the two forms. For clipped  $\tau_e$  strictly between zero and one,

$$w_e = -\ln(\tau_e)$$

$$\sum_{(e \text{ in path } \pi)} w_e = -\ln(\prod_{(e \text{ in } \pi)} \tau_e)$$

Since the logarithm is monotonic, minimizing the sum of transformed costs is equivalent to maximizing the product of edge probabilities. The edge-probability cap is therefore not just defensive programming: it prevents  $\ln(0)$  and preserves a finite nonnegative cost. If an implementation also includes a global multiplier in its route notion, it can multiply or otherwise reflect that factor consistently before the transform; the critical property is that larger opportunity corresponds to lower cost.

The planner starts from the initial outbreak locations. It identifies a bounded set of significant non-infected population destinations—implemented as the largest qualifying destinations—then runs shortest-path search toward them. Each road that occurs on a selected route receives an occurrence count. Roads are ranked by this count; endpoints of frequently selected roads induce a node order.

```
for source in initialSources:
    for target in majorPopulationDestinations:
         $\pi$  = Dijkstra(G, source, target, w)
        for edge e in  $\pi$ : routeCount[e] += 1

edgeScore(e) = routeCount[e]
nodeScore(v) = sum of incident route counts
```

The resulting intervention targets repeatedly likely source-to-major-destination **model routes**, not every short path in the graph. This is a deliberately bounded computation. Routing to every vertex would be slower, less interpretable, and would overstate resolution that the synthetic road probabilities do not support. Conversely, restricting to a handful of major destinations makes the method sensitive to that selection rule; this is a known limitation documented in the strategy rationale.

## 18.6 F.6 Max-flow/min-cut: structural separation heuristic

The max-flow/min-cut method transforms the road graph into a symmetric residual-capacity network. Road capacities are structural proxies derived from road class or imported class proxy, not observed passenger counts. The source is an initially infected vertex; the target is prioritized as a hospital if the graph includes one, otherwise a large qualifying population destination. This target policy turns “cut a city in half” into a more concrete educational question: what small structural separation lies between the outbreak source and a major protected destination?

```
initialize residual capacity r(u,v) from symmetric road capacities
while an augmenting source-to-target path exists:
```

```

b = minimum residual capacity on the path
subtract b from forward residual arcs
add b to reverse residual arcs

```

```

S = vertices reachable from source in final residual network
cutEdges = original edges with one endpoint in S and one outside S

```

The max-flow/min-cut theorem guarantees that the final maximum flow equals the capacity of a minimum source–target cut under the defined capacities. The theorem does **not** say the selected roads are the epidemiologically optimal closures. Their meaning depends on the capacity construction and the choice of source/target. In this simulator, the method is an instructive containment heuristic: if a small boundary separates the outbreak from a protected region, a constrained cut can be compelling.

The capacity proxy is a major documented decision. Using capacity rather than transmission probability distinguishes the min-cut strategy from Dijkstra. Dijkstra follows high-probability routes; min-cut finds low-total-capacity separation. They may select different roads for a good mathematical reason. If the application simply used the same  $\tau_e$  value for both without an explicit conceptual distinction, the comparison would be less educational.

## 18.7 F.7 From rankings to recommendations

After each strategy produces candidate lists, the common budget filter returns a plan. The plan carries the strategy name, selected nodes, selected roads, aggregate cost, and rationale fragments. The simulation then computes outcome statistics. The recommendation layer takes the first few actions of the leading plan and combines action type, target label, budget cost, local hotspot score, and strategy-specific reason. This layer was intentionally separated from the planner so that a change in prose does not change mathematical selection.

The ranking-to-recommendation pipeline contains several defensible non-decisions. It does not merge all five rankings into an ensemble score, because that would introduce a sixth unvalidated algorithm. It does not secretly improve one strategy’s candidates using hotspot evidence, because evidence must be held fixed and common. It does not automatically execute a real closure. It only describes a model action under an academic scenario.

## 18.8 F.8 Comparative interpretation matrix

|                                              |                       |                                            |                                  |                                     |
|----------------------------------------------|-----------------------|--------------------------------------------|----------------------------------|-------------------------------------|
| Strategy                                     | Structural hypothesis | Primary candidate score                    | Typical strength                 | Characteristic failure mode         |
| — — — —                                      | Random                | No graph signal is used                    | Seeded shuffle position          | Required neutral baseline           |
| Luck can mimic insight in one small scenario | Highest Degree        | Local hubs concentrate opportunity         | Incident-edge count              | Very fast, intuitive hub protection |
| Misses low-degree bridges                    | Betweenness           | Shortest-path bridges mediate connectivity | Brandes shortest-path dependency | Identifies topological gateways     |
| Shortest paths need not match movement       |                       |                                            |                                  |                                     |

| Dijkstra | High-product-probability routes matter | Repeated transformed shortest-route occurrence | Uses edge opportunity weights | Depends on chosen destinations and probability proxy | | Max-Flow/Min-Cut | A small capacity boundary can contain spread | Residual-network cut membership | Explains structural separation | Depends on capacities and source–target pair |

The matrix is the central teaching value of the project. A winning strategy is not announced as globally best. It is presented as the method whose structural hypothesis produced the lowest modeled outcome under one fully specified graph, parameter vector, budget, evidence set, and deterministic trial protocol.

## 19 Appendix G — Engineering design review and module-level rationale

This appendix records software-engineering decisions separately from mathematical decisions. A project can have correct formulas but still fail as a learning tool if its state is inconsistent, imports are unbounded, a desktop build silently depends on cloud services, or a visual animation changes the result it is meant to explain. The design adopted a full-stack TypeScript application with a shared domain layer, a browser-facing React workspace, a server-side API layer, optional persistence, and an Electron desktop wrapper. Each boundary was chosen to keep the calculation traceable rather than to maximize architectural novelty.

### 19.1 G.1 Layered architecture and the shared-domain decision

The source tree divides responsibilities into a client, a server, and a shared domain layer. The client owns interaction state and renders the workspace. The server owns API procedures, persistence adapters, bounded import calls, and explanation requests. The shared layer owns graph types, graph builders, epidemic functions, intervention planners, demonstration presets, and fixture-loading contracts.

```
client UI → typed API boundary → server services / storage adapter
      ↓
    shared graph and model contracts
      ↓
deterministic engine, planners, fixtures, test modules
```

The alternative would have been to implement the algorithms directly inside React components. That was rejected for several reasons. React render functions can re-run frequently; an algorithm executed during render could produce unstable state or wasted computation. More importantly, a component-local implementation is difficult to test without a browser and difficult to reuse in the desktop server. Pure shared modules make it possible to unit-test graph construction, infection transitions, strategy planning, fixture retrieval, and 3D profile mapping independently of the visual presentation.

The shared layer also prevents semantic drift. A `CityGraph`, `CityNode`, road edge, scenario parameter vector, intervention action, and outcome summary each have one primary TypeScript contract. The chart, 2D network explorer, 3D miniature, CSV export, PDF export, and share page all consume the same interpretation. That choice is more valuable than it appears: a mismatched edge identifier between a map and an algorithm would cause a closure to be shown but not applied, which is exactly the class of hidden error an academic simulator should avoid.

## 19.2 G.2 React workspace and the five-step navigation model

The application uses a desktop-first workspace that guides a user through Setup, Network, Outcomes, Comparison, and Analysis. This was chosen instead of a single dense dashboard because the project has multiple conceptual stages. A student first specifies a scenario, then inspects the graph, then watches a run, then compares strategies, then interprets a sensitivity or hotspot analysis. A one-screen dashboard would show all controls at once but would obscure this dependency order.

The sequence is guidance, not a restrictive wizard. State remains available across steps, and the user can return to a prior stage. This is important for an exploratory discrete-mathematics project: inspection of an unexpected result may legitimately lead back to the network view or parameter controls. The interface therefore uses a visual ribbon and progress context rather than a modal workflow that locks the learner into a linear path.

The design is explicitly desktop-first. The user requested no mobile-specific layout target, so engineering effort concentrated on a wide workspace with simultaneous controls, charts, and graph canvas. This is a prioritization decision, not a claim that small-screen use is impossible. Basic responsive behavior can still exist through the component system, but no feature is validated as a mobile product requirement.

## 19.3 G.3 State ownership, reset semantics, and side-effect control

The Home workspace owns scenario configuration, graph choice, active intervention plan, playback state, view choice, and derived result selection. Pure calculation functions receive snapshots of this state rather than mutating UI-owned objects. The reset control deliberately restores a single standard teaching scenario. It does not merely clear fields, because an empty state is a poor starting point for a classroom demonstration and cannot establish whether the system returned to a known configuration.

The reset is a substantive product decision. A reset must restore graph identity, parameter defaults, budget defaults, initial source choice, manual actions, timeline playback, sensitivity settings, active comparisons, and display messages in a coherent order. If it reset only the sliders while retaining stale manual closures or a previous imported graph, it would create an invalid apparent baseline. The associated rendered tests verify that this workflow stays stable.

React effects are used for asynchronous import completion, saved-scenario loading, and controlled UI synchronization rather than calculating core algorithmic outputs in an effect loop. This helps prevent the familiar bug where a new object literal on each render triggers an endless query or recomputation. The state model avoids setting state during render and uses memoization for derived collections where identity stability matters.

## 19.4 G.4 2D graph explorer: faithful interaction before visual decoration

The 2D explorer provides a legible graph-level view. Nodes and roads have consistent semantic colors, roads can show closure state, selected elements expose details, and pan/zoom make a moderate graph inspectable. The 2D view was retained even after the 3D miniature was enhanced because it is the more faithful analytic display. Topology, intervention selection, and path structure can be read without occlusion or perspective distortion.

The graph interaction model uses a transform consisting of translation and scale. Wheel input modifies zoom within a bounded range; pointer drag changes pan; selection is a separate click interpretation. These rules are tested because graph interaction bugs are deceptive: a failure to reset a transform can make it appear that a node disappeared, while the underlying graph data is correct.

The decision not to use a full geographic GIS library for the graph explorer is deliberate. The project needs a teaching graph with imported coordinates as an option, not turn-by-turn navigation or tile-based cartography. A bespoke SVG/canvas-style graph interaction is lighter, easier to test, and avoids an unrelated map-rendering dependency taking over the project.

## 19.5 G.5 3D architectural miniature: explanatory realism, not photorealism

The 3D view uses React Three Fiber and supporting Three.js utilities to render a high-detail architectural miniature. Building footprints, rooflines, streets, parcel surfaces, buffers, district lighting, outbreak halos, and quarantine fencing provide spatial cues. Facility type is expressed through differentiated geometry. Roads use surfaced strips and markings rather than bare graph lines. The tabletop camera creates a composed “city model” impression rather than a first-person scene.

This upgrade was a response to a user request for a prettier map, but its implementation is constrained by the academic purpose. Photorealism would make an invented synthetic population and probability field look too authoritative. The miniature style signals that the city is a model while still making hubs, corridors, blocks, and interventions intuitively visible. The 2D view remains available to guard against a 3D scene becoming the only evidence.

The 3D component delegates building-profile and facility geometry rules to a pure model module. That extraction is not merely code cleanup. It makes visual semantics testable: a hospital profile, an office profile, a road surface, or a quarantine fence can be validated without spinning up WebGL. It also keeps React rendering concerns from being entangled with the model’s classification logic.

## 19.6 G.6 Data import: bounded OpenStreetMap access and semantic transformation

The real-road option imports a deliberately bounded network from OpenStreetMap. The importer is not a general geographic-data pipeline. It validates a compact request, uses a small search region, applies endpoint failover, caches results, reduces source geometry to bounded nodes and edges, classifies road style, and fails explicitly when the network would be too large. This protects both classroom response time and the user from an opaque “loading forever” experience.

Imported road geometry must not be confused with imported disease data. OpenStreetMap supplies road-network geometry and road-class metadata. It does not supply household population, infection prevalence, movement intensity, clinical outcomes, or policy authority. The application therefore generates or asks for synthetic model inputs after geometry import. This separation is prominently documented because a real-looking map can lead users to infer that every displayed number is real.

An endpoint failover policy was selected because public OSM services can be rate-limited, unavailable, or temporarily slow. It is a resilience layer, not a guarantee. Error messages distinguish unavailable service, unsupported/oversized result, and invalid response. The test suite covers importer pending and error states; live importer verification is opt-in because a public service changes independently of source code.

## 19.7 G.7 Demonstration fixtures and offline-first teaching reliability

The original curated scenarios use bounded real-road extracts from East Village and Shoreditch. A prior desktop failure exposed an important delivery lesson: a Demonstration button that reaches a live network endpoint is not a demonstration mode. The corrected architecture stores curated road-graph snapshots as JSON fixtures and exposes a shared fixture loader. The client calls that loader for each preset instead of issuing an OpenStreetMap request.

The fixture import is bundled into the client-side build. Electron packaging validation searches for stable preset and graph identifiers in the produced assets rather than assuming that a source export name survives minification. This detail matters because a minified bundle can legitimately rename symbols; the relevant verification is that actual fixture content is present. The final validation confirmed fixture preset data in the packaged assets and verified that no Vite development-plugin reference leaked into the production archive.

The alternative of copying a fixture at application startup was less reliable, because it would create a path-resolution dependency inside an ASAR archive. Bundling it as a client import makes the demonstration graph available wherever the front-end bundle runs. The fixture is intentionally limited to curated teaching scenes; it is not an offline substitute for arbitrary map import.

## 19.8 G.8 Server procedures, persistence, sharing, and explanation boundaries

The server exposes typed procedures for durable scenario records, share identifiers, reports, bounded import work, and an optional plain-English explanation. Typed procedures were preferred to ad hoc untyped fetch calls because the same schema is validated across client and server. Public share pages carry a share identifier rather than exposing account or database internals. A user can create a reproducible view of a scenario without granting edit access to every reader.

The explanation endpoint deliberately receives a compact structured summary rather than the entire graph or a raw model trace. This decision has three motivations: a long graph payload adds latency and can exceed service limits; a decision-maker explanation requires conclusions and assumptions rather than every node; and a compact request reduces the chance that a generative explanation invents a detail from a vast irrelevant payload. The server prompt constrains language toward plain English and marks results as academic scenario projections.

Explanations remain secondary. The primary result, metrics, selected actions, raw plan, and formula-driven hotspot scores are produced by deterministic application code. An LLM cannot change the winning plan, fabricate an intervention cost, or provide a medical diagnosis. If the explanation service is unavailable, the simulator still runs and presents its non-generative evidence.

## 19.9 G.9 Export decisions: CSV, PDF, and permanent share link

The export design offers three different artifacts because they serve distinct review contexts. A CSV captures strategy comparison data for spreadsheet checking. A client-side PDF captures the visible report-oriented interface for a teacher or presentation. A share page provides a persistent linked scenario for a viewer. Treating these as one feature would be misleading: a PDF is frozen visual evidence, a CSV is a data extract, and a share page is an interactive record.

PDF generation uses browser-side rendering tools so that the scenario visual can be captured without a separate server-rendering pipeline. The code includes a safe fallback/error message, because image capture can fail on constrained browsers or complex canvases. The project does not claim that a PDF is an archival medical report; it is a classroom export of a bounded model view.

## 19.10 G.10 Electron portable desktop package

The desktop application uses Electron's main process to launch a local production server, open a BrowserWindow, manage lifecycle, and select a portable data folder beside the executable. The application does not depend on the cloud database when running in portable desktop mode. A local JSON store replaces cloud persistence for scenarios and reports. This decision supports the user's preference for a Windows executable and reduces reliance on browser hosting for demonstrations.

Desktop packaging required several production-specific choices. Development Vite plugins must not be statically captured by production serving code, because the Electron package contains a

built client and server rather than a JSX-transform environment. The server static-serving module was corrected to avoid that production leak. A staging script copies the production server bundle and built client assets into an Electron runtime layout; electron-builder produces a Windows x64 portable artifact. Runtime smoke coverage verifies the local serving path, while package inspection validates archive content.

The final release documentation reports a portable executable checksum, ensuring that a user can distinguish the validated artifact from an earlier build. This is a modest but meaningful software-engineering control: it makes “the final file” a verifiable object rather than a vague instruction to download the latest attachment.

## **19.11 G.11 Security, privacy, and non-goals**

No real personal health data is collected or required. Real-road import requests are bounded geographic queries, while population and infection values are synthetic educational values. Scenario persistence stores project state, not identity-sensitive health records. The design intentionally avoids claims of regulatory compliance, clinical safety, or operational authorization.

The project also does not attempt continuous background surveillance, scheduled refresh, real emergency notification, payment handling, or a general-purpose city operations platform. Each omitted capability is a boundary choice. Adding it would introduce data-governance, authentication, reliability, and policy responsibilities far beyond the stated discrete-mathematics objective.

## **19.12 G.12 Engineering review conclusion**

The implemented architecture is intentionally modest where correctness and reproducibility matter, and richer where explanation benefits: a pure shared mathematical core, typed service boundary, guided desktop workspace, analytical 2D graph, expressive but model-like 3D city, bounded import, offline demonstrations, and a portable Windows package. The most consequential engineering decision is not a framework selection. It is the refusal to let interface state, live web services, generative prose, or rendering decoration silently alter the mathematical scenario being compared.

## 20 Appendix H — Verification catalog, quality evidence, and residual test boundaries

Software verification is not the same as asserting that an application “looks finished.” This project has deterministic algorithms, asynchronous imports, interactive graph transforms, user-entered boundaries, persistence paths, visual themes, desktop packaging, and a public-data dependency. A serious quality argument therefore requires several kinds of evidence. This appendix explains the test strategy, the properties checked, the release gate, and the limits of the evidence.

### 20.1 H.1 Verification philosophy

The testing approach follows a layered claim. A pure mathematical routine should be checked with unit tests that make its input and expected output explicit. A rendered interaction should be checked in a component test that invokes events and observes user-visible state. A packaging claim should be checked through staging/asset inspection and local runtime smoke coverage. A live external service should have a bounded opt-in verifier rather than being required for every deterministic source test.

| Layer                 | Main question                                            | Typical evidence                                       | Why this layer is necessary                          |
|-----------------------|----------------------------------------------------------|--------------------------------------------------------|------------------------------------------------------|
| Pure model            | Does an algorithm return the prescribed result?          | Unit tests for graph, SIR, planners, fixture loader    | Fast deterministic proof of local behavior           |
| Component interaction | Does a control update the intended view state?           | Rendered tests with clicks, values, pointer actions    | Catches wiring and state-ownership errors            |
| Service boundary      | Does a request fail safely and return shaped data?       | Router/import/error-path tests                         | Makes external failures understandable               |
| Integration/package   | Does the staged app contain the right runtime resources? | Production build, desktop smoke, archive/asset checks  | Detects missing files and dev-only leakage           |
| Human review          | Is a result legible and correctly framed?                | Screenshot inspection, QA matrix, documentation review | Prevents technically correct but misleading delivery |

No single layer substitutes for another. A 100% unit-tested intervention planner can still have a broken “Run comparison” button. A visually polished dashboard can still call an outdated routine. A portable file can be large and present but omit an offline fixture. The value of the release process is in crossing these boundaries deliberately.

### 20.2 H.2 Mathematical core obligations

The epidemic engine tests cover more than nominal curve generation. They exercise deterministic replay from the same seed, validity of count bounds, recovery/mortality ordering, source handling, intervention removal, no-infection boundary conditions, and timeline aggregation. Each of these corresponds to a formal property documented in Appendix E.

- P1. Same  $(G, \theta, \text{actions}, \text{seed})$  produces identical outcome trace.
- P2. For every population-bearing node,  $S+I+R+D$  remains equal to  $N$ .

- P3. Counts never become negative or exceed available population.
- P4. A closed road or quarantined node has no active transmission connector.
- P5. A zero baseline does not produce undefined containment.
- P6. A strategy comparison uses identical inputs except strategy selection rule.

The test suite also examines deterministic trial keys. A more ordinary `Math.random()` implementation could create plausible outputs but would not permit a fair outcome-by-outcome comparison. Reproducibility tests therefore verify repeated calls and seed-controlled variation. This is a pedagogical quality condition: a lecturer should be able to repeat a demonstration and get the same stated winner.

Graph-construction tests check node categories, generated locations, weighted edges, road distances, population counts, and bounded synthetic topology. These tests do not prove that a synthetic city is realistic; they prove it meets its own declared contract. The report maintains this distinction throughout: implementation correctness is not empirical calibration.

## 20.3 H.3 Strategy-specific obligations

Each intervention method has a testable structural expectation. The Random strategy must use a reproducible shuffle and return an eligible ordering. Highest Degree must score endpoint incidence correctly. Betweenness must identify expected bridge-like vertices on small hand-checkable graphs. Dijkstra must convert probabilities to nonnegative route costs and return roads on selected paths. Min-cut must create a valid residual process and identify crossing cut candidates. The common budget filter must not exceed road, node, or population limits regardless of candidate order.

An especially important test set addresses fairness metadata. The comparison result carries graph identity or fingerprint, initial infected identifiers, complete parameter values, budgets, seed, and deterministic-trial configuration. The goal is not merely to show a winner; it is to make a winner falsifiable. A reviewer can inspect whether two strategies really used the same scenario.

Tie handling has a dedicated assertion. If multiple strategies have the same primary and secondary outcome tuple, the output must say that the result is tied or inconclusive. It must not select the first array member and call it best. This is an example of a small UI condition with substantive methodological significance. The user specifically noticed that a false “Random wins” result would undermine credibility; the corrected behavior is tested rather than left as a visual convention.

## 20.4 H.4 Input boundaries, invalid values, and recovery states

The workspace controls are tested across valid, boundary, invalid, and recovery states. This is not busywork. The app is intended to let students explore slider combinations, budgets, infection levels, and manual actions. Without boundary tests, a negative budget or invalid population could propagate into a later algorithm where the resulting error becomes difficult to explain.

The test matrix distinguishes several classes of behavior:

| Input/interaction class | Required system response | Example | |—|—|—| | Valid normal case |  
 Update scenario and recompute or enable run | A legal transmission slider change | | Lower/upper  
 boundary | Clamp or present permitted endpoint | Probability at 0 or 1 | | Malformed or incomplete  
 | Keep stable state and show clear guidance | Empty import query or invalid parsed number | |  
 Resource violation | Refuse safely with reason | Imported network beyond cap | | Pending external  
 action | Indicate work without stale false success | OSM import in progress | | Recoverable service  
 failure | Preserve workspace and explain next step | OSM endpoint unavailable | | Reset/retry |  
 Return to a known coherent state | Standard teaching reset |

Manual what-if actions receive additional care. A manual closure is allowed to change a scenario result, but it must remain distinct from an algorithmic plan. Tests verify the control surfaces and state transitions, while the comparison workflow continues to use only its own selected plan. This prevents a stale manual action from silently contaminating a supposedly fair five-strategy result.

## 20.5 H.5 Timeline, sensitivity, and graph-interaction coverage

The animated timeline is tested at both ends: it must not advance beyond the final day, must handle a reset or replay correctly, and must display consistent boundary information. These are subtle bugs because animation often works during a short manual inspection but fails when a user rapidly clicks play, pauses at the end, switches scenarios, or starts a sensitivity sweep.

Sensitivity controls are similarly tested for parameter selection, range changes, playback state, and output rendering. The tests do not validate that a sweep is a scientific uncertainty interval—the report explicitly says it is directional—but they validate that the requested parameter actually drives the displayed model runs and that a user cannot place the UI into an impossible playback state.

The 2D network test set covers zoom, pan, selection, and reset. Pointer-driven graphs are particularly prone to regression after styling work: an overlay may intercept a click, a transform may persist into a new scenario, or a selected node could become inaccessible after panning. The tests target these behaviors at the rendered interaction level rather than assuming that component code inspection is enough.

## 20.6 H.6 Theme, document, export, and share coverage

Global dark/light themes are tested not only for an isolated toggle but also for persistence and cleanup of the document root class. A theme bug can be technically harmless yet highly visible in a polished classroom application: text can become unreadable if a semantic background token is paired with the wrong foreground token. The tests and shared theme context reduce this risk. Export features are bounded by their medium. CSV tests verify strategy-comparison serialization and expected fields. PDF export is implemented with browser-side rendering tools and must show a fallback when browser capture is unavailable. Share-page procedures check scenario identifiers and public-view retrieval semantics. The test claim is therefore “the application prepares and

returns these artifacts as designed,” not “every viewer’s printer, browser, or network will render them identically.”

## 20.7 H.7 Importer and public-service evidence

The OpenStreetMap importer is a deliberate external boundary. It has tests for request formation, bounded result transformation, oversized-network errors, cache use, fallback behavior, pending state, and error display. A live request test exists as an opt-in verification only. The reason is methodological: a live Overpass/OSM endpoint can change, rate-limit, or fail while the source code is unchanged. Requiring a live request for the whole release gate would make a deterministic code-release assertion dependent on public-service conditions.

This does not mean the importer is untested. It means the evidence is correctly classified. Mocked/service-contract tests establish local behavior; an opt-in live test establishes a time-specific integration observation; documented fixtures establish reproducible Demonstration Mode. The project never uses fixture success to claim that all OpenStreetMap requests will work, nor does it use a transient OSM outage to claim the simulation itself is broken.

## 20.8 H.8 Desktop-package validation

The desktop release has its own quality path. A production client build and production server bundle are staged into the Electron runtime folder. A local runtime smoke test validates that the launcher can serve the packaged application path. Archive inspection confirms that development-only Vite plugin references are absent. Client-asset inspection confirms that actual demonstration fixture/preset content is present after minification and packaging.

The last point was learned through a real release failure. Searching for a source identifier such as an exported constant is not sufficient, because minifiers can rename or remove that literal. The corrected validation searches for stable data values such as preset identifiers and graph-fixture content. This is a general lesson in packaged-artifact testing: test the observable resource necessary for behavior, not a fragile implementation symbol.

The final portable executable is accompanied by a recorded file size and SHA-256 digest. This supports a simple release check on Windows. It does not mathematically prove that all hardware, antivirus environments, or user permissions will behave the same way. A user-side Windows smoke test remains the most direct confirmation of the packaged user experience, and it is explicitly distinguished from source-level test success.

## 20.9 H.9 Release-gate evidence and limitations

At the quality-assured release gate, TypeScript checking completed without errors and the automated suite contained 62 passing tests across 24 test files, with one intentionally skipped opt-in live verifier. This is meaningful evidence, but it must be stated with precision. Passing tests show that the listed checks passed on the project environment at that time. They do not establish

clinical validity, predict future external-service availability, or prove absence of every possible defect.

The project's QA documentation includes a traceable matrix covering valid, boundary, invalid, and recovery states. It also records external-service boundaries and visual-review notes. The value of this documentation is that a reviewer can locate what was actually checked, what was deliberately left contingent, and why.

## 20.10 H.10 Residual risk register

| Residual risk | Why it remains | Mitigation in the delivered project | Appropriate next step | |—|—|  
 —|—| | Public OSM outage or rate limit | External services are not under project control | Endpoint failover, cache, bounded error message, fixtures | Monitor service policy; use local data source for a production system | | Browser PDF variability | Canvas/font/cross-origin behavior differs by environment | Client fallback and alternate CSV/share exports | Server-rendered report service if required | | Windows environmental differences | Antivirus, permissions, graphics drivers vary | Portable package, local data, release checksum, smoke procedure | Test on target hardware before a presentation | | Numerical model uncertainty | Parameters are illustrative and not calibrated | Prominent scenario-model labels and limitations | Calibrate and validate against approved data in future research | | Algorithm objective mismatch | Graph heuristics optimize proxies, not social policy | Five-way comparison and rationale disclosure | Multi-objective participatory optimization with governance review |

The verification conclusion is therefore intentionally modest and strong at the same time. The application has reproducible source-level evidence for its declared software behavior, an explicit package-validation path, and documented residual risks. It does not pretend that a test count turns an educational model into an operational epidemic-control system.

## 21 Appendix I — Alternatives considered and complete rationale ledger

An implementation record should not read as if every chosen method were inevitable. The project made a sequence of constrained decisions: it had to be mathematically rich enough for discrete mathematics, understandable in a classroom demonstration, capable of working on a synthetic graph and bounded real-road geometry, fair across five strategies, visually compelling, and deliverable as a portable Windows application. This appendix records the principal alternatives and why the final design chose a different path. “Rejected” does not mean “bad”; it means unsuitable for the stated scope, evidence, or teaching objective.

### 21.1 I.1 Problem-framing alternatives

| Decision area | Alternatives considered | Adopted decision | Rationale and boundary | |—|—|—|—| | Primary purpose | Forecasting, live surveillance, policy automation, teaching simulator | Academic decision-support scenario simulator | A second-year project can rigorously compare algorithms without claiming a real-world forecast | | Disease representation | Agent-level simulation, ODE compartments, network-constrained discrete counts | Discrete-time node-level SIR-with-deaths | Preserves graph theory and countable day updates while remaining explainable | | City data | Fully synthetic, fully real demographic/GIS data, hybrid | Synthetic populations over synthetic or bounded real-road topology | Geometry can be authentic without implying health data is real | | Intervention claim | “Optimal lockdown” recommender, ranked heuristic comparison | Five named established strategies under a shared budget | Avoids inventing an unvalidated universal optimization objective | | Output voice | Clinical/risk prediction language, neutral scenario-model language | Explicit academic-projection framing | Prevents a classroom result being mistaken for public-health advice |

The final framing is not a rhetorical disclaimer appended after the fact. It shapes the whole architecture. An operational forecasting system would need data governance, calibration, uncertainty estimation, external validation, and policy authority. The chosen project instead makes its assumptions visible and treats its outputs as conditional consequences of those assumptions.

### 21.2 I.2 Epidemiological-model alternatives

Continuous-time ODE SIR versus discrete time. A Kermack–McKendrick-style differential-equation model is mathematically elegant and useful for population-level analysis. It was not selected as the main engine because the project must show a user a daily animated city graph with count transitions at named locations. A difference equation makes day  $t$ , day  $t + 1$ , sliders, and timeline playback immediately inspectable. It also aligns naturally with bounded deterministic trial keys. The cost is that numerical behavior depends on the chosen daily time step; this is stated as a limitation.

Agent-based simulation versus node-level counts. An agent-based model could represent individual movement, household membership, and contact events. It was not selected because it would require thousands of simulated agents, more unobservable assumptions, slower rendering, and a complex fairness problem across strategies. Node-level counts preserve a person-conservation invariant and make budgets expressible in modeled location populations. The compromise is heterogeneity within a node: every person in a location shares the same compartment count dynamics.

SIS, SEIR, vaccination, age stratification, and reinfection. These are all defensible extensions. The project adopts a susceptible–infected–recovered–deceased bookkeeping structure without an exposed compartment or reinfection. This choice avoids adding parameters that cannot be justified by classroom input. An SEIR model would require an incubation distribution and transition timing; age stratification would require contact matrices; vaccination would require efficacy and uptake assumptions. The report documents these as future-work directions rather than pretending their absence is unimportant.

Independent route approximation versus probability summation. Summing  $q_1 + q_2 + \dots$  is simpler but can exceed one. The selected complement-product aggregation remains bounded and demonstrates a useful probability identity. It assumes independent simplified opportunities, which is not realistic at city scale, but it is a better teaching approximation than unbounded addition. Correlated exposure would require a different data model and an empirically justified dependency structure.

Stochastic versus deterministic transitions. The project wanted stochastic-looking counts but reproducible comparison. It chose deterministic pseudo-random Bernoulli and binomial decisions keyed by seed and entity. A fully random runtime draw would feel natural but could make the strategy winner change when a user repeats the same setup. A purely expected-value update would be deterministic but would yield fractional people or need awkward rounding. Keyed integer draws are the chosen middle ground.

## 21.3 I.3 Graph and spatial-data alternatives

Grid graph versus city-block synthetic graph. A rectangular grid would be easy to implement and analyze, but it would make every street look structurally similar and weaken the demonstration of hubs, bridges, routes, and cuts. The selected synthetic builder creates homes, schools, hospitals, offices, blocks, and weighted road types. Its visual vocabulary supports intervention narratives while remaining explicitly fictional.

One node type versus semantic node types. A generic graph is mathematically sufficient for degree or betweenness. Semantic node types were added because intervention cost and explanation need a vocabulary. Quarantining a hospital, school, or office can be described and assigned a modeled population cost; a road intersection remains a connector. The decision introduces more complexity but avoids presenting a location recommendation as an anonymous vertex number.

Raw OpenStreetMap geometry versus bounded reduced graph. Importing all nearby roads would maximize visual detail but could create enormous graphs, unstable queries, unresponsive centrality computation, and unreadable maps. The importer therefore reduces and bounds geometry. This is a loss of geographic fidelity, but it is an honest and necessary one. A teaching simulator benefits more from a graph that a learner can inspect than from a city dataset that overwhelms the interface.

Live map dependence versus fixtures. Initially, curated demonstrations could use live OpenStreetMap import. This failed the actual requirement of reliable demonstration. The final choice bundles East Village and Shoreditch fixtures. The cost is that fixtures become static snapshots; the benefit is that a presentation can be repeated offline. A full production mapping system would need versioned local datasets and update policies, but that was outside scope.

Geographic coordinates as epidemiological distance. The project uses road geometry to organize a graph, not to infer physical transmission distance. Edge probabilities are synthetic model values. A road's length or class may influence a weight or capacity proxy, but the report does not claim that this measures real contact probability. This decision preserves the distinction between topology and epidemiological calibration.

## 21.4 I.4 Intervention-algorithm alternatives

One composite “smart score” versus separate established strategies. A combined score could mix degree, betweenness, probability, and capacity. It was rejected because its weights would be arbitrary and its winner hard to explain. The final design compares five methods with clear hypotheses. A learner can say why a bridge heuristic differs from a route heuristic.

PageRank, eigenvector, closeness, or community detection. These graph methods were plausible candidates. They were not selected because the five adopted strategies already cover a baseline, local degree, global shortest-path bridging, weighted routes, and structural cuts. Adding more would make comparison tables crowded and weaken the ability to explain each method in a timed demonstration. The report identifies these methods as possible extensions, not omissions caused by ignorance.

Weighted betweenness versus unweighted Brandes. A weighted form could incorporate edge distances or probabilities. It was not adopted because Dijkstra is already the designated probability-aware method. Keeping betweenness unweighted isolates its topology-based bridge interpretation. This is a teaching design choice; it should not be interpreted as a claim that real mobility is unweighted.

Every-source/every-destination Dijkstra versus bounded major destinations. Running every pair would produce a more exhaustive route count but costs more and creates a less interpretable candidate objective. The final route method focuses on initial sources and the largest qualifying destinations. This bounds computation and supports a narrative of blocking likely routes toward major destinations. The selected destination set is therefore a model assumption and should be displayed, not hidden.

Global all-pairs min-cut versus one source–target min-cut. A global cut could identify community separation but raises ambiguity about which populations should be separated. The adopted source–target formulation chooses an initial outbreak source and a prioritized hospital/major destination. It gives a concrete theorem-driven example and keeps computation small. Its output is conditional on that source–target choice.

Soft intervention effect versus edge/node removal. An intervention could reduce transmission probabilities rather than close roads or quarantine locations. Complete removal was selected because the user explicitly requested “which roads to close” and “which buildings to quarantine,” and because it makes graph transformation visible. A partial-effect extension would be more realistic but would require effectiveness probabilities and compliance assumptions.

## 21.5 I.5 Fairness and outcome alternatives

Different budgets per strategy versus common constrained budget. Different budgets might allow each algorithm to show its best behavior, but such a comparison would be inequitable. The final system holds road, location, and population budgets common. Every strategy may propose many candidates, but the common filter turns candidates into comparable feasible plans.

Strategy-specific stochastic seeds versus common random numbers. Different seeds could sample a plausible range, but a difference in an individual comparison might be noise. The final protocol holds the deterministic seed and key scheme fixed. This supports a counterfactual reading: under the same synthetic chance realization, how does changing the intervention alter the trace? Multiple-seed uncertainty analysis is documented as future work.

Single scalar score versus lexicographic outcome comparison. An arbitrary weighted score could combine infection, deaths, cost, and duration. It was rejected because weights would hide a value judgment. The adopted order uses lower final cumulative infections first and lower modeled deaths second, while displaying other measures for inspection. This still contains a normative priority, but it is explicit and simple enough to debate.

Always name one winner versus support ties. A dashboard may be tempting to design around a single winner. The final rule allows ties and labels them as inconclusive. This is mathematically correct when outcome tuples coincide and academically healthier than claiming a spurious ranking.

Use Random only as hidden control versus show it prominently. Random is shown because it helps students evaluate whether a graph heuristic added value. Its presence is accompanied by explanation and tie safety. Hiding it would make the comparison look more sophisticated but less scientifically honest.

## 21.6 I.6 User-experience and visualization alternatives

Single dashboard versus guided five-step workspace. The guided workspace was selected to communicate a pedagogical sequence. A single dashboard would reduce navigation but overload

novices with controls. The chosen design preserves revisitation so it does not become a rigid wizard.

2D only versus 2D plus 3D. A 2D-only graph would be analytically clean and cheaper. The user requested a more impressive map, and the 3D miniature was added as a complementary contextual view. Retaining 2D was critical: a 3D scene should not be the only way to judge topology.

Photorealistic city versus architectural model. Full photorealism was rejected because it could imply geographical/epidemiological precision that the synthetic model lacks. The selected miniature uses richer architectural cues while maintaining a visibly constructed, tabletop character.

Manual actions mixed with algorithms versus separated what-if mode. Mixing manual closures into a strategy run would make outcomes hard to attribute. The final app lets a user explore manual actions but labels that path separately from the five-way comparison. This protects methodological fairness while keeping interactive play.

Plain raw output versus recommendations and plain-English explanation. Raw tables are necessary but not sufficient for a decision-support demonstration. The project adds recommendations with target, action, rationale, expected impact, and cost, plus a compact optional explanation. The explanation cannot alter the deterministic result and is framed in non-clinical language.

## **21.7 I.7 Persistence, export, and deployment alternatives**

Browser-only local state versus server persistence. Browser-only state is simple but makes sharing and scenario recovery weak. The web application includes server persistence and public share identifiers. The portable desktop mode substitutes a local JSON store because cloud database access is not required for an offline executable.

PDF only versus CSV and share link. PDF alone preserves presentation but not data; CSV alone preserves data but not visual context; a link supports interactive review but depends on a hosted path. The final three-mode export design recognizes all three uses.

Hosted web application versus Windows portable executable. The user ultimately preferred a desktop executable because a hosted website felt less dependable for presentation. The project therefore packages Electron Windows x64 portable output as the primary artifact. The web version remains useful for development and share pages, but the release framing privileges the portable demonstration.

Electron installer versus portable executable. A traditional installer could register an application and manage updates, but it adds permissions, installation friction, and presentation risk. The portable executable can be placed in a writable folder and run directly. The project records the caution that its companion data directory should be writable.

## **21.8 I.8 Documentation alternatives**

Short README versus long technical review. A README is efficient for setup instructions but cannot defend model assumptions, strategy fairness, implementation choices, QA evidence, and limitations. The requested report therefore acts as a full technical review. It includes formal

notation, derived formulas, worked numerical examples, decision tables, test evidence, and ethical limitations.

Invented academic citations versus carefully scoped references. The report uses canonical references for concepts such as SIR, shortest paths, centrality, and max-flow/min-cut, and distinguishes those from project-specific implementation decisions. It does not falsely imply that a source validates the project’s synthetic parameters. This is a documentation integrity decision.

Claim perfection versus record residual risk. The documentation explicitly names OSM dependency behavior, portable-Windows environmental differences, parameter uncertainty, and objective mismatch. This makes the report more credible than a blanket claim of being “bug-free.”

## 21.9 I.9 Ledger conclusion

The decision ledger has one repeated pattern. When an alternative would provide more realism, automation, or visual polish only by adding unvalidated assumptions or obscuring the discrete-mathematics lesson, the project selected the more explicit and bounded option. When an alternative would improve reproducibility or explainability without changing the pedagogical scope—such as shared domain types, keyed trials, fixture bundling, tie handling, capped imports, or a common budget filter—the project adopted it. This is the organizing principle behind the final artifact.

## 22 Appendix J — Worked mathematical case studies and audit calculations

This appendix supplies concrete arithmetic for readers who want to verify how the project's equations behave before opening the code. The values are intentionally small and illustrative. They are not copied from a real location, they are not a recorded simulation trace, and they do not represent a disease forecast. Their purpose is to show how the model's discrete mathematics, probability rules, graph transformations, and comparison protocol fit together.

### 22.1 J.1 Case study 1: typed graph and baseline population audit

Consider a teaching graph with six vertices:

```
h = home, n_h = 90
s = school, n_s = 180
o = office, n_o = 140
c = clinic, n_c = 70
j = road intersection, n_j = 0
p = road intersection, n_p = 0
```

The population-bearing set is  $V_P = \{h, s, o, c\}$  and the routing-only set is  $V_R = \{j, p\}$ . The modeled city population is therefore

$$N = 90 + 180 + 140 + 70 = 480$$

Suppose the roads are  $h - j$ ,  $j - s$ ,  $j - o$ ,  $s - p$ ,  $o - p$ , and  $p - c$ . The hand-checkable degree sequence is

$$d(h)=1, d(j)=3, d(s)=2, d(o)=2, d(p)=3, d(c)=1$$

This small example makes a basic but important point about typed graphs. The road intersection  $j$  has degree three, which may make it structurally important; it has zero modeled population, which makes it ineligible for a population quarantine in the selected constraint vocabulary. The school has degree two but carries 180 modeled people. A planner that used only degree would see  $j$  and  $p$  as strong local connectors. A planner that is subject to a population budget must also reason about the cost of any semantic location it recommends.

Let the initial state at the home be

$$X_h(0) = (S_h, I_h, R_h, D_h) = (82, 8, 0, 0)$$

and let all other population nodes start susceptible. Conservation holds at each vertex:

```

82+8+0+0=90
180+0+0+0=180
140+0+0+0=140
70+0+0+0=70

```

The city total also conserves people: 480 before and after a transition. The road intersections retain zero compartment counts. If an implementation accidentally initializes road vertices with a nonzero population or allows an infection count to arise there, the error is immediately visible in this table; it is not a subtle epidemiological disagreement.

## 22.2 J.2 Case study 2: one source, one route, one daily exposure

Let global transmission multiplier  $\alpha = 0.8$ . Give the road  $h - j$  base opportunity  $\tau_{h,j} = 0.06$ . The capped elementary opportunity is

$$x_{\text{--}}(h,j) = 0.8 \times 0.06 = 0.048$$

With  $I_{h(0)} = 8$  infected people in the source location, the engine's simplified independent-opportunity probability for at least one successful source-to-neighbor exposure is

$$\begin{aligned}
 q_{\text{--}}(h \rightarrow j) &= 1 - (1 - 0.048)^8 \\
 &= 1 - 0.952^8 \\
 &\approx 1 - 0.6747 \\
 &\approx 0.3253
 \end{aligned}$$

The number is not “32.53% of real commuters become infected.” It is an event-level probability proxy in the educational engine. Its function is to demonstrate two formal properties. First, it rises as source infection count rises. Second, it remains below one without a special corrective rule. The same calculation with one infectious person gives 0.048; with eight simplified opportunities it is approximately 0.3253, not 0.384, because the events can overlap.

Assume for a further illustrative step that the intersection leads to the school through another road and the model's aggregation produces school exposure probability  $p_{s(0)} = 0.20$ . The school has  $S_{s(0)} = 180$ . Its number of new infections is a binomial draw with support from zero to 180:

$$\begin{aligned}
 Y_{\text{--}}s &\sim \text{Binomial}(180, 0.20) \\
 E[Y_{\text{--}}s] &= 180 \times 0.20 = 36 \\
 \text{Var}[Y_{\text{--}}s] &= 180 \times 0.20 \times 0.80 = 28.8
 \end{aligned}$$

The calculation does not mean every run produces 36 new cases; it describes the expected value of the model transition. In the deterministic replay design, the keyed pseudo-random draws will return one particular integer between zero and 180 for a specified seed. Repeating the same

scenario with the same seed returns the same integer. Repeating it with a new seed produces a different legitimate scenario trace.

## 22.3 J.3 Case study 3: multiple sources and the complement product

Suppose the school has two independent simplified exposure channels on a day. One source produces  $q_1 = 0.3253$  and the other produces  $q_2 = 0.1800$ . The aggregate risk is

$$\begin{aligned} p_s &= 1 - (1-q_1)(1-q_2) \\ &= 1 - (0.6747)(0.8200) \\ &= 1 - 0.5533 \\ &= 0.4467 \end{aligned}$$

Naively adding the two probabilities would give 0.5053. That number is not necessarily impossible here, but it double counts cases in which both channels succeed. The complement product avoids the double count under the model's independence assumption. It also gives the correct boundary behavior: if either  $q_i = 0$ , it has no effect; if any  $q_i = 1$ , the aggregate is one; and the result cannot exceed one.

The approximation's limitation is visible as well. The two sources may share people, travel patterns, or contextual factors, so their exposure events need not be independent. A correlated model would require joint probabilities or a latent-contact process. Those inputs are not present in this project. The complement rule is selected because it is mathematically transparent, bounded, and appropriate for an introductory graph-and-probability demonstration.

## 22.4 J.4 Case study 4: recovery and mortality ordering

Return to the home with  $I_{h(0)} = 8$ . Let daily recovery parameter  $\rho = 0.125$  and mortality parameter  $\mu = 0.02$ . The expected recovery count is

$$E[R_{h^+}] = 8 \times 0.125 = 1$$

Imagine that the keyed trial sequence returns exactly one recovery. The eligible infected count for mortality is then seven, not eight. The expected mortality conditional on that recovery count is

$$E[D_{h^+} \mid R_{h^+}=1] = 7 \times 0.02 = 0.14$$

If the key sequence yields zero deaths, the home's updated state before considering any incoming new infection is

$$\begin{aligned} S_h(1) &= 82 \\ I_h(1) &= 8 - 1 - 0 = 7 \\ R_h(1) &= 0 + 1 = 1 \\ D_h(1) &= 0 + 0 = 0 \end{aligned}$$

Again  $82 + 7 + 1 + 0 = 90$ . If new infections at the home were allowed from other sources, they would appear through the separately calculated  $I_h^+$  term; in this simple initial-source case we take it as zero. The sequential order has a clear logical advantage. No infected individual is simultaneously added to recovered and deceased counts. A model that sampled both from the original eight could need an artificial correction when  $R_h^+ + D_h^+ > 8$ .

## 22.5 J.5 Case study 5: road closure as a graph transformation

Take the six-road graph described in Case J.1. Consider a road action  $A_E = \{h - j\}$ . The resulting active edge set is

```
E_active = {j-s, j-o, s-p, o-p, p-c}
```

If  $h$  is the only initial infected location, it has degree zero in the active graph. Under the engine's connector-based exposure rule, it cannot transmit to the rest of the graph on that day. This is not a claim that one real road closure would isolate real household contacts. It is the exact counterfactual defined by this graph scenario.

Now consider location action  $A_V = \{s\}$ , a school quarantine. The school remains drawn in the interface with a quarantine marker, but its usable transmission connectors are removed. The active graph is conceptually

```
V_active = {h, j, o, c, p}
E_active = {h-j, j-o, o-p, p-c}
```

The display decision—show the school while making it inactive—lets a reviewer see the selected policy action. Removing it both visually and mathematically could make a reader wonder whether it was ever part of the graph; leaving edges visually active would be worse, because it would contradict the calculation.

## 22.6 J.6 Case study 6: budget feasibility with population constraints

Suppose a planner ranks school  $s$  first, office  $o$  second, clinic  $c$  third, then roads. Let the budget be

```
B_v=2 locations
B_p=200 modeled people
B_e=2 roads
```

The school costs 180 people. It is feasible, so it is selected. The office costs 140 more people;  $180 + 140 = 320 > 200$ , so it is skipped even though it is ranked second. The clinic costs 70 more people;  $180 + 70 = 250 > 200$ , so it is skipped. The node plan contains only the school, even though the count budget could accommodate one more node.

Next, suppose the ranked roads are  $j - o$ ,  $o - p$ ,  $h - j$ . The first two are selected because  $B_e = 2$ . The feasible plan is

```
V_A={s}; E_A={j-o,o-p}
|V_A|=1≤2
Σ N_v = 180≤200
|E_A|=2≤2
```

This example explains why “top two nodes” is not a valid description of the actual planner. The plan is selected in ranking order **subject to feasibility**. It also reveals an intentional limitation: the greedy filter may leave unused node-count budget because no remaining low-cost candidate fits the population budget. A global knapsack optimizer might choose a different combination. The project prefers a common, explainable greedy feasibility rule over a new hidden optimization layer.

## 22.7 J.7 Case study 7: degree and betweenness diverge

Build a graph of two triangles connected by a single low-degree bridge. Let left triangle vertices be  $a, b, x$ , right triangle vertices be  $y, c, d$ , and bridge edge be  $x - y$ . The degrees are

```
d(a)=2, d(b)=2, d(x)=3, d(y)=3, d(c)=2, d(d)=2
```

Degree appropriately highlights  $x$  and  $y$ , but consider a slightly richer left cluster in which an internal hub  $h$  has degree five while  $x$  remains degree three. Highest Degree may choose  $h$ . Betweenness will often favor  $x$ , because every shortest route between the left and right clusters uses the bridge endpoints. If a closure budget allows one node, the difference becomes visible: one method protects a locally dense connector; the other protects an inter-community gateway. The formula for betweenness counts proportional shortest-path participation. If there are  $m$  ordered source–target pairs that require  $x$ , then each contributes one or a fraction to  $C_{B(x)}$ . An internal hub may have higher local degree but fewer cross-cluster dependencies. Neither method is universally correct; the graph’s shape decides which structural hypothesis is more useful.

## 22.8 J.8 Case study 8: Dijkstra transformation of a transmission route

Consider two candidate paths from source  $h$  to destination  $c$ .

```
path A: opportunities 0.40, 0.50 => product 0.20
path B: opportunities 0.70, 0.20 => product 0.14
```

Without a transform, products are awkward for standard shortest-path accumulation. Apply  $w = -\ln(\tau)$ :

```
cost(A) = -ln(0.40) - ln(0.50) = -ln(0.20) ≈ 1.6094
cost(B) = -ln(0.70) - ln(0.20) = -ln(0.14) ≈ 1.9661
```

Dijkstra chooses path A because  $1.6094 < 1.9661$ , which is equivalent to its greater opportunity product  $0.20 > 0.14$ . The transform is mathematically exact for positive multiplicative edge opportunities. It is not an arbitrary score conversion. The project then counts roads that appear on several selected source-to-major-destination routes and prioritizes repeated bottlenecks.

This case also explains the cap near one. If  $\tau = 1$ , then  $-\ln(1) = 0$ , which is mathematically finite but can create zero-cost chains; if  $\tau = 0$ , then  $-\ln(0)$  is undefined/infinite. Capping and validating input gives the algorithm a stable finite cost domain.

## 22.9 J.9 Case study 9: a small max-flow/min-cut construction

Let an outbreak source  $s$  connect to two intermediate nodes  $a, b$ , both of which connect to protected destination  $t$ . Give capacities

$$c(s,a)=3; c(s,b)=2; c(a,t)=1; c(b,t)=2$$

The maximum possible flow is at most  $1 + 2 = 3$  because the final two incoming capacities to  $t$  sum to three. A feasible flow of three exists: send one through  $s - a - t$  and two through  $s - b - t$ . Therefore the maximum flow equals three. The cut containing  $s, a, b$  on one side and  $t$  on the other has capacity  $1 + 2 = 3$ , so it is a minimum cut.

In the simulator's teaching interpretation, roads  $a - t$  and  $b - t$  form a capacity-defined separation boundary between source region and protected destination. This does not mean physically closing the two roads is necessarily socially acceptable or epidemiologically optimal. It means the theorem identifies the lowest-total-capacity boundary under supplied capacities. If road capacities are only class proxies, the output must be interpreted as structural evidence, not a field measurement.

## 22.10 J.10 Case study 10: Bayes update from symptom evidence

Suppose a location has prior infection probability  $P(H) = 0.10$ . Let reported symptoms  $E$  have likelihood  $P(E|H) = 0.70$  under infection and  $P(E|\neg H) = 0.20$  without infection. Bayes' theorem gives

$$\begin{aligned} P(H|E) &= [0.70 \times 0.10] / [0.70 \times 0.10 + 0.20 \times 0.90] \\ &= 0.07 / 0.25 \\ &= 0.28 \end{aligned}$$

The posterior is 0.28, higher than the prior 0.10 because the evidence is more likely under the infection hypothesis. The project's hotspot scoring uses a bounded, transparent evidence update for triage/explanation. It is not a medical diagnostic model: the likelihoods are scenario inputs, symptoms are not calibrated clinical evidence, and a posterior score cannot diagnose any individual. The main pedagogical value is to show how a prior and evidence likelihood combine, and how the resulting score can be displayed alongside graph structure without replacing it.

## 22.11 J.11 Case study 11: containment and tie-aware comparison

Assume the no-intervention baseline ends with cumulative infections  $C_0 = 150$ . Two strategies produce

```
Degree: C_D=105, modeled deaths=4
Betweenness: C_B=105, modeled deaths=4
Random: C_R=110, modeled deaths=5
```

The degree containment fraction is

$$\eta_D = (150-105)/150 = 0.30$$

and betweenness has the same  $\eta_B = 0.30$ . Since the primary and secondary outcome tuple is identical, the result is a tie. The interface should say “inconclusive tie between Highest Degree and Betweenness” rather than choose whichever list item appears first. Random is not best merely because a sort implementation preserves it in some order; its primary outcome is worse.

If the baseline had  $C_0 = 0$ , the containment fraction is explicitly reported as zero rather than evaluating a fraction with zero denominator. That branch reflects a logical interpretation: no baseline infections means no intervention can take credit for preventing a modeled outbreak.

## 22.12 J.12 Case study 12: finite-difference sensitivity reading

Let an outcome statistic  $Y(\alpha)$  be final cumulative infections under multiplier  $\alpha$ . Suppose the model gives  $Y(0.70) = 80$  and  $Y(0.80) = 105$ . The forward finite-difference sensitivity around 0.70 is

$$\Delta Y / \Delta \alpha = (105-80) / (0.80-0.70) = 25 / 0.10 = 250$$

This says that in this discrete sweep interval, a one-unit change in the multiplier would correspond to approximately 250 outcome units if the local linear approximation remained valid. It does **not** claim that the true derivative exists or remains 250. The engine uses integer outcomes, thresholds, caps, and pseudo-random trial comparisons; the curve can be non-smooth. The sensitivity panel therefore correctly describes a directional parameter sweep, not a calibrated confidence interval.

## 22.13 J.13 Case-study conclusion

The calculations demonstrate the project’s central methodological promise. Every displayed recommendation can be traced to a finite graph, a budget-feasible action, a defined probability rule, a deterministic scenario key, and an explicit comparison tuple. The numbers are conditional, not prophetic. Their academic value comes from being inspectable, repeatable, and challengeable.

## 23 Appendix K — Complete module, data-flow, and review audit

This appendix is a map for a technical reviewer. The aim is not to reproduce every source line; source control already preserves that. The aim is to state what each important module is responsible for, which data it accepts, what it returns, what assumptions it is allowed to make, and what a reviewer should inspect if behavior appears incorrect. A useful technical review follows data through the system, rather than treating files as an unstructured list.

### 23.1 K.1 Domain layer inventory

| Module or contract area | Main responsibility | Inputs | Outputs | Review invariant | |—|—|—|—|  
 | Shared types | Defines CityGraph, nodes, edges, parameters, plans, outcomes | Type declarations  
 | Common compile-time vocabulary | Client and server agree on identifiers and optional fields | |  
 Synthetic graph builder | Creates a bounded teaching city | Seed/configuration | Typed city graph  
 with semantic nodes and weighted roads | IDs unique; road endpoints exist; population valid | |  
 Epidemic engine | Calculates daily count transitions | Graph, parameters, actions, seed | Timeline,  
 totals, derived metrics | Per-node conservation and deterministic replay | | Intervention planner |  
 Ranks candidates and returns feasible actions | Graph, source/evidence context, budget | Plan and  
 explanatory rationale | Does not exceed road/node/population budget | | Demonstration presets |  
 Defines curated teaching scenario inputs | Preset identifier | Graph fixture choice and parameters |  
 Each preset is reproducible and intentionally named | | Fixture loader | Supplies offline road graphs  
 | Demonstration preset | Bundled graph snapshot | No live OSM request for curated demo | | 3D  
 model rules | Maps graph semantics to geometry profiles | Node/edge metadata | Pure building/  
 road visual profiles | Visual type matches domain type |

The shared-type decision is a form of executable documentation. If a field is renamed or narrowed in the domain model, the compiler surfaces client/server uses that need review. This is safer than duplicating a `nodeId`, `roadId`, or `strategy shape` in several unrelated files. It does not eliminate all logic bugs, but it prevents a large class of integration mistakes before runtime.

### 23.2 K.2 Scenario creation data flow

The following flow applies when a user starts from the standard synthetic teaching scenario.

1. Workspace selects standard graph configuration.
2. Synthetic builder generates semantic nodes and roads with stable IDs.
3. Setup controls supply  $\theta$ : transmission, recovery, mortality, horizon, budget, seed.
4. Initial infection choice supplies  $X(0)$  or source identifiers.
5. Optional symptoms/evidence supply hotspot inputs.
6. User requests a baseline or five-way comparison.

7. Shared engine/planner return immutable result objects.
8. Client renders timeline, graph state, table, recommendation cards, and analysis.

Each arrow is a possible review boundary. For example, if a slider visually changes but the comparison result does not, inspect the conversion from control state to parameter vector. If a selected node is marked but the timeline remains unchanged, inspect the action-to-active-graph transformation. If CSV contains different totals from the comparison card, inspect the outcome-summary serialization. The data flow makes fault isolation more systematic than general debugging.

The builder uses deterministic construction rules so the standard teaching case can be reset. Generated geometry is a model aid. Node coordinates help render a city-like shape and roads, but they do not change the fact that locality and population are synthetic. A reviewer should verify that all edge endpoints resolve to known node identifiers and that road class/weight fields have admissible values before asking algorithmic questions.

## 23.3 K.3 Real-road import data flow and explicit semantic gap

The bounded OpenStreetMap importer follows a distinct path.

```
request parameters → input validation → cached response lookup
  → endpoint attempt / failover → raw OSM way/node data
  → bounded graph reduction → normalized CityGraph
  → synthetic population/model enrichment → workspace scenario
```

The semantic gap occurs between normalized road geometry and synthetic epidemiological enrichment. An imported road node or edge may preserve coordinate and road-class information, but it does not become a real household, school population, infection record, or mortality observation. The software must not fill that gap silently with a claim of factual realism.

The importer's cap is an important resource invariant. A request that would exceed the maximum manageable node/edge count is rejected with a specific oversized-network result. This is better than allowing centrality or 3D rendering to freeze. The cache avoids repeated identical public queries and helps recover from transient endpoint conditions. Failover is used only after a recognized failure; it is not a looping retry mechanism that could overload an external service.

For review, inspect three different classes of importer result: a normal compact graph, an oversized error, and an unavailable-service error. Each should leave the prior workspace scenario stable. A failed import must not replace a known teaching graph with a half-parsed network.

## 23.4 K.4 Engine input/output audit

At a run boundary, the engine should be treated as a pure function conceptually equivalent to

```
simulate(G, θ, initialState, actionSet, evidence, seed) → RunResult
```

The actual TypeScript call structure may use multiple helper functions, but the data-contract idea is stable. `RunResult` contains a sequence of daily counts, node-level or graph-level state as needed by the visual layer, cumulative values, peak values, duration, and comparison-ready metrics. It should not contain mutable React state, DOM references, arbitrary API responses, or user-interface strings. Keeping result data plain makes it serializable for persistence, sharing, CSV, and the local desktop store.

The engine receives actions as inputs; it does not call the intervention planner while transitioning each day. This is a major causal boundary. A plan is selected once from the pre-run context, then applied as a fixed scenario intervention. If the engine replanned every day without the interface making that policy explicit, it would be simulating adaptive control rather than a single budget-constrained intervention. Adaptive policies are valuable research topics, but they are not what the five named strategies are intended to compare here.

Review questions for the engine include the following.

| Review question | Correct evidence location | Typical failure symptom | |—|—|—| | Are people conserved? | Daily transition unit tests and node counts | Total population drifts | | Are actions applied? | Active-graph helper and closure tests | Closed road still transmits | | Are outputs repeatable? | Seed/key tests | Same run gives different winner | | Is time synchronous? | Transition-order code and multi-node test | Node ordering changes outcome | | Are probabilities bounded? | Input clamps and exposure helper | NaN, negative, or >1 risk | | Is baseline isolated? | Comparison runner | Baseline contains a prior plan |

## 23.5 K.5 Planner input/output audit

The planner consumes the graph and scenario context but returns candidates before final plan selection. This enables each strategy to expose **why** it ranked a node or edge. For example, an output can say that Highest Degree selected a connector based on incident roads, Betweenness selected a bridge based on shortest-path dependence, Dijkstra selected a repeated likely route road, or Min-Cut selected a boundary edge.

The common feasibility function is a reusable guard rather than copied logic. Its input is an ordered candidate list and budget triple. Its output must be a subset of candidates. It should never create actions that were not ranked, and it must produce aggregate cost accounting. A reviewer should test both an ample-budget case, in which early candidates all pass, and a constrained case, in which a high-population node is skipped and a later node may be considered.

The recommendation writer consumes a plan and result. It does not invent new locations. A textual explanation can say “quarantine the selected school because it lies on many shortest paths,” but it cannot replace the selected school with a more rhetorically appealing target. This separation between computation and prose extends to optional LLM explanations.

## 23.6 K.6 Demonstration mode audit

The curated Demonstration Mode is a teaching product, not an implicit benchmark. Its five scenarios are chosen so that the five strategies each have a context in which their structural hypothesis produces a leading result under the defined fairness protocol. The scenarios are named to make the intended lesson visible: chance alignment, hub protection, bridge protection, route blocking, and cut separation.

Demonstration flow is

Demonstration button

- choose curated preset
- fixture loader resolves offline graph
- preset applies parameters, seed, budget, sources, evidence
- comparison executes through normal common runner
- presenter notes and expected teaching narrative appear

The key review invariant is that the preset does not call the live OSM importer. A web browser or desktop app may have no network connection during a presentation. Fixture content must therefore be part of the compiled client assets. The package verification used stable embedded preset/fixture data, not a source-level export-name search, because minification can change symbol names.

The second invariant is transparency: curated winner scenarios are not evidence that a strategy wins generally. The presenter notes explain that the scenario was constructed to illustrate a particular structural condition. The ability to return to the standard teaching scenario and adjust values restores exploration after the scripted demonstration.

## 23.7 K.7 Visualization and interaction audit

The 2D and 3D views are both projections of the same graph/result state. Neither owns an independent disease model. Selection in either view may update UI focus; it should not mutate infection counts. A visual outbreak halo must derive from the current state; it should not be a random decorative animation whose intensity contradicts the displayed counts. Quarantine fencing must derive from a selected node action; it must not appear merely because a node is a school.

The 2D transform state includes scale and translation. The 3D state includes camera configuration and controlled scene effects. Each should be reset or reinitialized appropriately when the graph changes. Reviewers should check: selection matches stable identifiers; closed roads show a changed style; pan does not lose selection identity; zoom bounds prevent unusable scale; theme switches do not erase semantic colors; and animations do not change core results.

The decision to keep a 2D analytical mode is a validation control. A reviewer can compare the 3D miniature's apparent geography with a simple topological drawing. If an intervention seems visually surprising, the 2D graph often makes its degree, bridge, or route context easier to inspect.

## 23.8 K.8 Persistence, sharing, and local desktop audit

For the hosted mode, scenario records and reports move through typed server procedures to persistence. A share identifier exposes a restricted public representation, not raw database access. The record must retain enough information to reconstruct the scenario: graph identity or serialized graph data, parameters, action context, comparison result metadata, and creation details. A share view should be treated as a snapshot, not as a dynamically re-running live external import. For desktop mode, a local file-based store replaces cloud database operations. The principle is to keep the portable executable usable when no hosted services are available. Local data should reside beside the portable application in a writable directory. The design does not promise enterprise backup, multi-user synchronization, or secure medical record storage. It persists educational scenarios and reports for a single portable user context.

Data-flow review should ensure that serialization preserves deterministic fields. If a saved scenario loses its seed, it may rerun to a different trace. If a share view loses manual-action distinction, it may misrepresent a fair strategy comparison. The contract must therefore serialize the scenario description, not only a human-readable summary.

## 23.9 K.9 Explanation and export audit

The optional explanation path begins after deterministic results exist. It receives a concise structured payload of leading strategy, results, budget, selected actions, major assumptions, and audience instructions. It returns prose for nontechnical decision-makers. This is intentionally a one-way explanatory flow:

```
engine / planner result → compact explanation payload → explanatory text
```

There is no backward arrow from explanatory text into an action plan. If the explanation endpoint is unavailable, user-visible deterministic cards still supply outcomes and rationale. Reviewers should test an explanation failure path for a graceful message and ensure that a large graph payload is not unnecessarily sent.

The CSV path serializes comparison metrics; the PDF path captures presentation content; the share path retrieves a persistent view. These exports should name model projections clearly. In particular, a CSV column such as “modeled deaths” must not be renamed “actual deaths” during export. Semantic wording is a data-contract requirement, not a cosmetic preference.

## 23.10 K.10 Build, package, and release audit

The production release pipeline can be understood as a sequence of transformations:

```
TypeScript source
→ type check and unit/rendered tests
→ production client build and server bundle
→ desktop runtime staging
```

- Electron builder portable Windows artifact
- archive/content verification and checksum
- user-device smoke test

Each stage proves a different thing. Type checking proves contract consistency, tests prove specified behavior, a production build proves bundling compatibility, staging proves resource arrangement, the Electron artifact proves a distributable file exists, content verification proves critical assets are present, and a Windows smoke test proves the user-facing environment launches. These must not be collapsed into “build succeeded.” A build can succeed while missing a fixture; a fixture can be present while an antivirus policy blocks launching; a launch can work while a live import endpoint is unavailable.

The project previously corrected a production issue in static serving by preventing a Vite development plugin from being imported in the desktop production path. This illustrates why release builds deserve their own review. Development servers often mask conditions absent from the packaged environment. The final inspection verifies that no development plugin reference remains in the packaged archive and that the offline demonstration assets are embedded.

## 23.11 K.11 Reviewer walkthrough

A technical reviewer can follow this compact walkthrough.

1. Open the standard scenario and note graph, parameter, budget, seed, and source values.
2. Run baseline and five-way comparison; record the shared metadata.
3. Inspect each strategy plan and verify action costs do not exceed budget.
4. Use 2D view to inspect selected roads/nodes; compare to the strategy rationale.
5. Play timeline to verify finite daily evolution and totals.
6. Switch to 3D view as a contextual visualization, not as independent evidence.
7. Try a manual closure and confirm it is separate from strategy results.
8. Run a curated offline demonstration without requesting live map import.
9. Export CSV/PDF or open share view; verify scenario-model language is retained.
10. Reset to the standard scenario; confirm no stale actions or transforms remain.

The walkthrough demonstrates not just feature availability but the intended separation of concerns. It asks the reviewer to test critical boundaries where a product might otherwise look correct while silently mixing data, action states, or environments.

## 23.12 K.12 Final audit conclusion

The system is organized around an inspectable flow: typed scenario description to pure model calculation, strategy candidate ranking to common feasible plan, result to visualization/export/explanation, and source build to validated desktop artifact. This organization is the practical complement to the mathematical specification. It gives a lecturer, examiner, or future maintainer a way to answer the most important technical question: **when the screen says a method is**

**leading, which graph, assumptions, budget, rule, and packaged resource produced that claim?**

## 24 Appendix L — Proof sketches, complexity bounds, and numerical design compendium

This final technical appendix gathers the core formal arguments in one place. The project is a software implementation, not a theorem paper, so these are proof sketches tied to the implemented assumptions. Their purpose is twofold. First, they show why the calculations preserve the properties the interface claims. Second, they make the computational cost of each choice visible, which is necessary when a graph is imported or when a classroom user changes a parameter and expects an interactive response.

### 24.1 L.1 Proposition: population conservation is preserved

Claim. If a population-bearing vertex starts day  $t$  with nonnegative integer compartment counts satisfying

$$S + I + R + D = N,$$

and if new infection count  $Y$  satisfies  $0 \leq Y \leq S$ , recovery count  $Z$  satisfies  $0 \leq Z \leq I$ , and death count  $W$  satisfies  $0 \leq W \leq I - Z$ , then the sequential update

$$\begin{aligned} S' &= S - Y \\ I' &= I + Y - Z - W \\ R' &= R + Z \\ D' &= D + W \end{aligned}$$

produces nonnegative integer counts that also sum to  $N$ .

Proof sketch. Integers remain integers because subtraction and addition of integers preserves integrality. Since  $Y \leq S$ ,  $S' = S - Y \geq 0$ . Since  $Z + W \leq I$ ,  $I - Z - W \geq 0$  and thus  $I' = I - Z - W + Y \geq 0$ . The recovered and deceased updates add nonnegative values. Summing the four equations cancels  $-Y$  with  $+Y$ ,  $-Z$  with  $+Z$ , and  $-W$  with  $+W$ , leaving  $S + I + R + D = N$ . The argument applies independently at every population-bearing node. Summing across nodes proves city-wide conservation as well.

This proof identifies why transition order matters. If deaths were sampled from the initial  $I$  after recoveries were sampled from that same  $I$ , the condition  $Z + W \leq I$  would not automatically hold. The selected sequential availability rule provides exactly the premise required by the proof.

### 24.2 L.2 Proposition: elementary and aggregate exposure probabilities are bounded

Claim. For  $0 \leq x \leq 1$  and nonnegative integer  $m$ , the quantity

$$q = 1 - (1-x)^m$$

lies in  $[0, 1]$ . If  $q_1, \dots, q_k$  all lie in  $[0, 1]$ , then

$$p = 1 - \text{product over } i \text{ of } (1 - q_i)$$

also lies in  $[0, 1]$ .

Proof sketch. For  $x$  in the unit interval,  $1 - x$  is in the unit interval. Raising a unit-interval value to nonnegative integer power remains in the unit interval. Its complement is therefore also in the unit interval. For aggregate exposure, every complement  $1 - q_i$  is in the unit interval; the finite product is in the unit interval; subtracting that product from one leaves a value in the unit interval. No post hoc clipping is needed to make the probability valid, although the implementation also validates input before evaluation.

The result has useful monotonicity. Increasing any one  $q_i$  cannot reduce  $p$ , because it reduces one factor in the nonnegative product. Increasing source infection count  $m$  cannot reduce  $q$  for fixed  $x$ , because  $(1 - x)^m$  is nonincreasing in  $m$  when  $0 \leq 1 - x \leq 1$ . These claims match the intuitive direction of the visual risk display without claiming that the probabilities are calibrated observations.

## 24.3 L.3 Expected value and variance of the binomial transition

Let  $Y$  denote the new-infection count at a susceptible pool of size  $S$  under daily probability  $p$ . Under the engine's independent-trial approximation,

$$\begin{aligned} Y &\sim \text{Binomial}(S, p) \\ E[Y] &= S p \\ \text{Var}[Y] &= S p (1-p) \end{aligned}$$

The expectation follows by writing  $Y = X_1 + \dots + X_S$ , where  $X_i$  is an indicator for person  $i$  receiving the simplified event. Linearity of expectation gives  $E[Y] = \sum E[X_i] = Sp$ . Under independent indicators, variance adds and each indicator has variance  $p(1 - p)$ , giving  $Sp(1 - p)$ . The implementation returns an integer keyed draw rather than the expectation; the expectation nevertheless explains the average direction of changes in a parameter sweep.

The standard deviation is

$$\text{SD}[Y] = \text{squareRoot}( S p (1-p) )$$

For a small illustrative pool with  $S = 180$  and  $p = 0.20$ , the expected count is 36 and the standard deviation is approximately 5.37. A single deterministic trace might return 31 or 42 without contradicting the model. The scenario's seed makes the realized integer reproducible; it does not remove the conceptual stochasticity represented by the binomial formulation.

## 24.4 L.4 Proposition: active intervention graph is a subgraph

Claim. Given original graph  $G = (V, E)$  and action sets consisting of selected road closures  $E_A$  and quarantined vertices  $V_A$ , the active graph  $G_A$  has vertex and edge sets that are subsets of the original graph's vertex and edge sets.

$$V_{A\_active} \subseteq V$$

$$E_{A\_active} \subseteq E$$

Proof sketch. A closure action removes an existing edge from consideration; it never adds an edge. A quarantine action removes or disables a known vertex's usable incidence; it never introduces a vertex or road. Therefore every traversable node and edge after action existed before action. The proof is simple, but its product implication is important: an intervention cannot create a new transmission path through a graph bug. The graph view may retain a quarantined vertex for visual annotation, but the transmission adjacency structure observes the subgraph rule.

A related monotonicity statement deserves careful wording. Removing edges cannot create a new simple path that did not already exist. It can, however, change which remaining path is shortest or most probable among survivors. Thus an intervention can change Dijkstra route ranking without violating subgraph monotonicity. This is one reason the planner selects a plan before the simulation rather than dynamically re-ranking every day without disclosure.

## 24.5 L.5 Budget feasibility theorem for the common filter

Claim. If a greedy budget filter begins with zero selected node count, road count, and selected population; only adds a node when doing so would not exceed  $B_v$  or  $B_p$ ; and only adds a road when doing so would not exceed  $B_e$ , then its output plan is feasible.

Proof sketch by induction over candidate order. Initially, all three resource sums equal zero and are feasible because budgets are nonnegative. Assume the selected prefix is feasible. For the next candidate node, the filter either skips it, preserving feasibility, or adds it only after checking that node count plus one is at most  $B_v$  and population plus its nonnegative cost is at most  $B_p$ . For the next road, the filter skips it or adds it only when road count plus one is at most  $B_e$ . Hence feasibility remains true after each candidate. By induction, the final plan is feasible.

The theorem proves feasibility, not optimality. Greedy ranking plus budget acceptance does not solve a general joint integer optimization problem. That distinction is intentionally preserved in the interface and report. The planner's result means "the highest-ranked feasible prefix under this strategy and filter," not "the globally optimal policy."

## 24.6 L.6 Complexity of graph construction and one daily engine step

Let  $n = |V|$ ,  $m = |E|$ , and  $T$  be number of days. A bounded synthetic graph builder typically runs in time proportional to the number of nodes and constructed roads, so  $O(n + m)$ . Memory is also  $O(n + m)$  for node/edge records and adjacency representation.

One daily transition requires traversing active connectors to aggregate source opportunities and visiting population-bearing nodes to apply count transitions. With adjacency lists, the dominant graph scan is  $O(n + m)$  in the usual bounded model representation. Over a horizon of  $T$  days, the direct engine cost is

$$O(T(n+m))$$

plus the cost of individual simplified binomial trial sampling if that is implemented literally as one loop per available person. Let  $P = \sum N_v$  be total modeled population. In the worst literal trial implementation, a daily update can include  $O(P)$  indicator work, giving  $O(T(n + m + P))$ . The project caps teaching graph size and modeled quantities accordingly. A future high-scale system would use a more efficient binomial sampler rather than iterating every conceptual person.

This estimate explains two product choices. First, imported graphs are bounded before they reach repeated algorithms. Second, the application uses scenario-level populations suitable for a teaching simulator, not millions of individual agents. These are performance and scope decisions, not assertions that a real city has only a few hundred people.

## 24.7 L.7 Complexity of Highest Degree and Random

Degree computation requires one pass over edges, incrementing endpoint counters. Its time is  $O(m)$  and its storage is  $O(n)$ . Sorting candidates is  $O(n \log n)$  for nodes and  $O(m \log m)$  for roads. The total is therefore dominated by sorting after linear scoring:

$$O(m + n \log n + m \log m)$$

Random's seeded shuffle is linear after arrays are constructed, typically  $O(n + m)$ , followed by the common linear scan/filter. It does not need centrality, shortest paths, or residual networks. The project includes it precisely because it provides a low-computation non-structural reference. Stable tie-breaking by identifier is a small computational decision with outsized reproducibility value. Equal degree or equal pseudo-random selection position should not become environment-dependent array-order behavior. Stable order makes a saved scenario reviewable and prevents presentation differences caused by a nonsemantic sort implementation.

## 24.8 L.8 Complexity and correctness intuition of Brandes betweenness

For unweighted graphs, the Brandes method runs breadth-first search from each source. One BFS plus accumulation is  $O(n + m)$ . Repeating it for all sources yields

$$O(n(n+m))$$

with  $O(n + m)$  working memory per source for predecessor lists, distances, path counts, and stack/queue structures. This is dramatically better than enumerating every source-target pair and all their shortest paths naively, which can be exponential in the number of equal shortest paths. The correctness intuition is dependency accumulation. During BFS from source  $s$ , each reached vertex has a distance and a count of shortest paths from  $s$ . Processing vertices in reverse distance order lets a vertex distribute dependency back to its predecessors in proportion to their shortest-path counts. Every downstream shortest path contribution is thus credited to upstream vertices that lie on those paths. Summing over sources yields the standard centrality quantity.

The project's use of unweighted topology deliberately limits computational and conceptual scope. A weighted Brandes variant would need a priority queue per source and a weighted path-count procedure. This is possible, but it would duplicate the probability-aware route concept already introduced through Dijkstra and make a real-time teaching graph less responsive.

## 24.9 L.9 Dijkstra correctness under probability transformation

Dijkstra's algorithm returns shortest paths when all edge weights are nonnegative. With a valid opportunity  $\tau_e$  in the open interval  $(0, 1]$ , define  $w_e = -\ln(\tau_e)$ . Because  $\ln(\tau_e) \leq 0$ ,  $w_e \geq 0$ . Thus the nonnegative-weight requirement holds.

For a path  $\pi$ ,

$$\begin{aligned} & \text{sum of } w_e \text{ over } e \text{ in } \pi \\ &= \text{sum of } -\ln(\tau_e) \\ &= -\ln(\text{product of } \tau_e) \end{aligned}$$

The negative logarithm is strictly decreasing on positive values: if product A is larger than product B, then negative-log A is smaller than negative-log B. Therefore a path minimizing transformed sum maximizes the product of simplified edge opportunities. Dijkstra is correct for the transformed route criterion, assuming its usual graph and weight prerequisites.

Using this proof does not validate the epidemiological meaning of multiplying edge probabilities. That assumption comes from the educational scenario model. The proof only establishes that, **given that assumption**, the chosen graph algorithm optimizes the stated path criterion exactly. With a binary heap, each run is commonly bounded by  $O((n + m) \log n)$ . If the planner runs it from  $r$  selected sources to target computations reusing distance maps where possible, the practical

cost depends on implementation detail. The project bounds source/destination subsets to keep the route-blocking strategy responsive under classroom graph sizes.

## 24.10 L.10 Max-flow/min-cut correctness intuition and cost

The max-flow/min-cut implementation uses a residual network. An augmenting path carries additional flow equal to its minimum residual capacity. Updating forward and reverse residual capacity preserves flow conservation at intermediate vertices and records the possibility of canceling prior choices. When no source-to-target augmenting path remains, the set of vertices reachable from source in the residual network defines a cut. No original residual-positive edge can cross from reachable to unreachable; otherwise the target side would be reachable. The saturated original crossing edges form a capacity boundary.

The max-flow/min-cut theorem states that the total value of the maximum flow equals the total capacity of a minimum source–target cut. The project relies on this theorem as a structural containment heuristic. It does not extrapolate the theorem to social policy: capacity values are proxies, and the source/target pair is a scenario choice.

Runtime depends on the augmentation implementation and capacities. A straightforward Ford–Fulkerson-style method has weak general bounds for arbitrary real capacities. With integer capacity proxies and breadth-first augmenting paths, an Edmonds–Karp-style behavior has a classical  $O(nm^2)$  bound. At the project’s bounded graph scale, this is acceptable for a small number of selected source–target computations. The importer cap and target prioritization are therefore tied directly to algorithmic cost.

## 24.11 L.11 Finite-difference sensitivity and derivative interpretation

The sensitivity display varies one input while holding the remaining scenario description fixed. For output statistic  $Y(\theta)$  and small step  $h$ , it can display a finite difference

$$[Y(\theta+h) - Y(\theta)] / h$$

as a numerical local slope indicator. In a deterministic discrete-count simulator, this should not be mistaken for an analytic derivative. Integer outcomes can jump at thresholds, budget feasibility can change discontinuously, and selected actions may change rank when parameter differences alter route scores. A single slope is therefore descriptive, not a proof of smooth response.

The fixed-input rule is nevertheless crucial. If the sweep also changed the seed, initial source, graph, or budget, a chart’s variation would mix causes. Holding those values fixed lets a student say: within this modeled scenario, what changes when the selected parameter changes? This is a controlled computational experiment, not a calibrated uncertainty analysis.

## 24.12 L.12 Deterministic hash-trial properties

Let  $H(k)$  map a composite key  $k$  to a finite unsigned integer space, then normalize  $U(k) = \frac{H(k)}{M}$  for maximum space size  $M$ . A deterministic Bernoulli function is

$$B(k, p) = 1 \text{ if } U(k) < p; \text{ otherwise } 0$$

For any fixed key and probability, the result is fixed. If  $p_1 \leq p_2$ , then  $B(k, p_1) = 1$  implies  $B(k, p_2) = 1$ ; this monotonicity comes from the same uniform threshold value. The property can make certain comparative changes less noisy: increasing a probability does not randomly reverse an already successful trial for the same key.

The technique does not make hash outputs independent in a strict statistical sense, and the project does not claim a cryptographic random source. Its rationale is counterfactual comparability. The same scenario state receives the same trial threshold across strategy runs. If an action disconnects an exposure, the action's graph effect—not a new random number—is what changes the outcome opportunity.

## 24.13 L.13 Numerical safeguards and their formal role

| Safeguard | Mathematical risk avoided | Why it is not merely interface polish | |—|—|—| | Probability clamp to unit interval | Invalid Bernoulli/domain values | Preserves probability-space assumptions used in proofs | | Edge opportunity cap below one | Infinite or degenerate transformed route cost | Keeps Dijkstra weights finite and useful | | Nonnegative budget clamp | Impossible resource arithmetic | Establishes base case for feasibility induction | | Integer time horizon | Undefined timeline indices | Defines finite sequence of daily states | | Population availability bounds | Negative compartment counts | Supplies premises for conservation proof | | Zero-baseline containment branch | Division by zero | Gives a defined semantic result for no-outbreak case | | Stable tie rule | Nonsemantic ordering effect | Prevents false unique winner claim |

The table illustrates a broader engineering lesson. Input validation is part of the mathematical model. A proof that uses probabilities in  $[0, 1]$  cannot be separated from code that ensures values stay in that domain.

## 24.14 L.14 What is proved and what is not

The report can legitimately claim the following conditional statements: count conservation follows from the stated update and bounded draws; aggregate probabilities are bounded under the stated independence approximation; Dijkstra minimizes the transformed additive route cost; max-flow/min-cut identifies a minimum cut under supplied capacities; budget filter output is feasible; deterministic keys reproduce identical inputs.

The report does not prove that real infections follow independent trials, that OpenStreetMap road class is a contact capacity, that a school closure is ethically desirable, that a generated population is demographically accurate, or that a winning graph heuristic is a real-world policy optimum.

These are empirical, ethical, or governance claims requiring evidence beyond the source code. Explicitly naming the boundary is part of the mathematical rigor of the project.

## **24.15 L.15 Compendium conclusion**

The calculations and proofs in this appendix show that the simulator's internal rules are coherent at the level claimed: finite graphs, bounded probabilities, conserved model populations, reproducible trials, feasible budgets, and established graph-algorithm objectives. The same analysis also explains why the application remains an academic scenario tool. Formal internal coherence is necessary for a useful model; it is not sufficient for real-world predictive or policy authority.

## 25 Appendix M — Reviewer traceability matrix and technical-defense guide

This appendix turns the report into a usable examination document. It identifies the claim a reviewer may hear, the implementation evidence that supports it, the qualification that prevents overclaiming, and a concise way to demonstrate the claim in the application. It is deliberately included because an academic project should be defensible under questions, not merely attractive during a silent demo.

### 25.1 M.1 Claim-to-evidence matrix

| Project claim | Technical evidence | Demonstration method | Required qualification | |—|—|—|  
 —| | The city is modeled as a graph | Typed nodes/edges, adjacency, 2D/3D views | Select a node and inspect connecting roads | Synthetic graph semantics are not a cadastral city record |  
 | Disease progression is discrete-time SIR-style | Compartment transition engine and timeline |  
 Step one day, inspect S/I/R/D totals | It is a scenario model, not a clinical forecast | | Strategies are compared fairly | Shared runner, same seed/parameters/graph/budget | Run five-way comparison and inspect metadata | Equal conditions do not make strategies universally equivalent | | The plan observes budget | Common feasibility filter and cost display | Lower budget and observe skipped candidates | Greedy feasibility is not global optimization | | Dijkstra identifies likely routes | Negative-log edge transformation | Compare a high-product route to a low-product route | Opportunity values are modeling proxies | | Min-cut identifies a containment boundary | Residual network and cut-edge output | Load cut-separation demonstration preset | Capacity is structural proxy, not a real closure order | | Bayesian scoring updates a prior | Evidence controls and posterior hotspot score | Change symptom evidence and observe score | Not diagnosis or clinical triage |  
 | Real-road input is bounded | OSM importer caps/cache/error states | Request an oversized or unavailable import path | Road geometry is real when imported; population/disease values are synthetic | | Demonstrations work offline | Bundled fixtures and preset loader | Start a curated scenario without map request | General live OSM import still requires connectivity | | Desktop package is portable | Electron portable artifact and checksum | Run from writable Windows folder | User-device security policies can differ | | Quality checks exist | Type check, test suite, QA matrices | Review validation documents and rerun suite | Passing tests do not prove absence of all defects |  
 This matrix is a safeguard against a common presentation failure: replacing a precise claim with a grander but less defensible claim. For example, the statement “the simulator predicts a pandemic” is neither needed nor supported. The statement “the simulator projects finite academic scenarios under explicitly displayed assumptions” is accurate and technically stronger.

### 25.2 M.2 Requirement-to-module traceability

| Requirement family | Primary modules | Secondary modules | Observable artifact | |—|—|—|—|  
 Synthetic city graph | Graph builder, shared types | 2D/3D views | Resettable standard scenario |

| Real-road exploration | OSM importer | Cache, error UX | Import panel and provenance notices  
 | | SIR progression | Epidemic engine | Timeline panel | Daily state chart and totals | | Five inter-  
 ventions | Planner | Comparison scorecard | Strategy columns and selected actions | | Fairness |  
 Comparison runner | Seed/key helper | Same-condition metadata | | Budget constraints | Feasibility  
 filter | Recommendations | Road/node/population costs | | Bayesian evidence | Hotspot calculation  
 | Analysis panel | Prior/evidence/posterior explanation | | What-if actions | Workspace controls |  
 Active graph | Manual closure/quarantine labels | | Sensitivity | Parameter sweep controls | Charts  
 | Controlled input-outcome trace | | Persistence/share | Server routes/local store | Share page |  
 Saved scenario/public view | | PDF/CSV export | Export components | Summary data contract |  
 Downloadable artifacts | | Light/dark interface | Theme context | All view components | Global  
 color mode | | Desktop delivery | Electron main/runtime staging | Local storage | Portable Windows  
 executable |

Traceability prevents “feature drift.” A feature cannot be considered complete merely because a button exists. It must map to a module that changes state or produces a defined result and to an artifact a reviewer can inspect. The matrix also makes maintenance more manageable: when a shared type changes, it identifies the UI, import, engine, and export surfaces likely to require review.

## 25.3 M.3 Mathematical-defense questions and model answers

Question: Why use SIR-like compartments rather than a full agent-based disease model?

The discrete compartment system is selected for an undergraduate discrete-mathematics project because it exposes state transitions, graph adjacency, probability, and counting in an inspectable form. A full agent-based model would require individual attributes, contact schedules, calibration data, and performance controls. The educational model still supports the relevant formal questions: how does a transition preserve counts; how does probability accumulate through a graph; and how do graph actions change available connectors?

Question: How can a stochastic model be fair if different strategies could get lucky?

The comparison locks the graph, initial state, parameters, horizon, budget, and seed. Keyed deterministic trials ensure that equal opportunities use comparable threshold values. Thus, when a strategy produces a different result under the same scenario description, the difference is attributable to altered graph connectivity and action selection, rather than an unrelated fresh random sequence. The report does not claim that one deterministic trace estimates an expected population outcome; it claims counterfactual comparability.

Question: Why are probabilities combined with a product rather than summed?

For independent simplified opportunities, the probability that none occurs is the product of their complements. Taking one minus that product yields the probability that at least one occurs. Direct addition is only valid in special disjoint-event conditions and can exceed one. The independence

assumption is a model simplification, but the complement identity is mathematically bounded and correct under that stated simplification.

Question: Why does the Dijkstra transform contain a logarithm?

Transmission-route opportunity is represented multiplicatively along a route, but Dijkstra solves additive nonnegative shortest-path problems. The map  $w = -\ln(\tau)$  converts a product into a sum. Minimizing the sum is equivalent to maximizing the positive product because negative logarithm reverses order. The transform is therefore a derivation, not a hand-tuned score.

Question: Does Max-Flow/Min-Cut tell a city which roads to close?

No. It identifies a low-capacity boundary in the supplied graph for chosen source and target regions. The output is a structural candidate for discussion. The capacity inputs are proxies, and actual closure decisions require legal, social, mobility, health-service, and empirical information outside the model.

Question: Why not declare a winner whenever one strategy sorts first?

Sorting order may contain arbitrary stable implementation effects. The comparison uses an explicit outcome tuple and detects equal outcomes. A tie-aware result is more honest. It teaches that graph heuristics can be indistinguishable under a particular scenario and that evidence is conditional on the chosen metric.

## 25.4 M.4 Engineering-defense questions and model answers

Question: Why use a shared domain layer?

Graph nodes, actions, parameters, outcomes, and strategy identifiers are consumed by the client, server, simulation engine, fixtures, exports, and desktop runtime. Shared types make their agreement a compile-time property. Without them, an import response might use one road identifier shape while the 3D scene or export expects another, producing subtle runtime errors.

Question: Why have both a web application and a desktop executable?

The web application supports development, public sharing, and browser-oriented workflows. The portable executable is the primary final delivery because a presentation environment may be offline, unfamiliar, or sensitive to hosted services. Electron packages the production client/server runtime with local persistence. The two routes share model code, reducing divergence.

Question: Why bundle only demonstration fixtures instead of all possible map data?

Curated fixtures solve a specific offline reliability requirement. Bundling arbitrary city data would inflate the executable and create stale-data/provenance questions. The application retains a bounded live importer for exploratory use when network access exists, while the teaching presets use a compact, verified offline set.

Question: Why separate optional LLM explanation from computation?

An explanation service can make deterministic results easier to understand, but it should not decide the graph plan or change outcome counts. Separating it makes its failure nonfatal, limits payload size, and makes the causal chain auditable. The outcome exists before explanatory prose is generated.

Question: Why verify package contents rather than trust a successful build?

A build process can produce an executable that launches yet lacks a runtime asset needed for a particular branch. The offline fixture issue illustrates the difference. Checking the packaged client bundle for stable fixture/preset content gives an additional release gate; an end-user Windows smoke test remains the final environment check.

## 25.5 M.5 Formal review checklist

A reviewer who wants to audit the mathematics can apply this list.

| Check | Method | Expected result | |—|—|—| | State total | Sum S, I, R, D at a node before/after day | Same modeled population | | Probability bounds | Test  $p=0$ ,  $p=1$ , multiple sources | Output remains in  $[0,1]$  | | Recovery/death availability | Use small I with high parameters | No negative infected count | | Action feasibility | Set low budgets | Plan cost never exceeds caps | | Edge removal | Close bridge road | No path can use that road | | Degree ranking | Use known star/hub graph | Hub is ranked above leaves | | Betweenness | Use two clusters with bridge | Bridge ranks highly | | Dijkstra transform | Compare route products | Higher product has lower transformed cost | | Min-cut | Use small capacity network | Cut equals max-flow value | | Bayesian update | Evidence more likely under H | Posterior exceeds prior | | Tie handling | Construct equal outcomes | Result is a tie, not arbitrary winner |

The checklist is intentionally a mixture of algebraic boundary tests and graph-structure tests. An epidemic graph simulator can fail either way: its formulas may be correct while the graph action is wrong, or its graph action may be correct while the transition violates conservation.

## 25.6 M.6 Presentation sequence for a 12–15 minute review

1. Begin with the decision problem: limited interventions on a connected city graph.
2. State the model boundary: academic scenario projections, not forecasts.
3. Show the graph contract and explain typed locations/roads.
4. State the daily compartments and one conservation equation.
5. Show the five strategy families and their distinct structural hypotheses.
6. Explain fairness controls before showing any winner.
7. Load a curated offline demonstration and run a comparison.
8. Inspect the selected actions in 2D view, then use 3D view for spatial context.
9. Change an input or manual action to show what-if behavior.
10. Show sensitivity and evidence scoring as controlled auxiliary analysis.
11. Demonstrate export/share or desktop portability.
12. End with limitations, ethical boundary, and future validation work.

This order is defensible because it moves from assumptions to algorithms to evidence to limitations. Starting with an attractive 3D scene may engage an audience, but it risks making visualization look like the project's core proof. The mathematical and experimental controls should arrive before the animated result.

## 25.7 M.7 Final reviewer note

The project's strongest defensible contribution is not the claim that it has discovered a new containment algorithm. It is a carefully integrated educational decision-support simulator that makes established discrete-mathematics tools comparable under shared conditions. The report records each major choice, its alternative, its computational consequence, and its limitation. That is the standard against which the project should be reviewed.

## 26 Glossary of terms and notation

$C_0$  – **baseline cumulative infections**: The final cumulative infection count with no algorithm-selected intervention.

$C_{B(v)}$  – **betweenness centrality**: The sum of fractions of shortest paths between pairs that pass through a vertex.

$B$  – **intervention budget**: Upper bounds on closed roads, quarantined nodes, and quarantined population.

$c(e)$  – **edge capacity**: A modeled road-class proxy used only by the max-flow/min-cut strategy.

$\eta$  – **containment rate**: The bounded relative reduction in final cumulative infections against the baseline.

$D$  – **deceased population**: People removed through the model's mortality transition.

$\deg(v)$  – **degree centrality**: The number of graph edges incident to a vertex.

$e$  – **edge**: A road segment joining two vertices.

*Electron* – **Electron desktop shell**: The desktop runtime that launches the portable Windows application and local loopback server.

*fairness protocol* – **fair comparison protocol**: Shared graph, outbreak parameters, initial infections, seed, duration, and budget across all five strategies.

$P(E|\neg H)$  – **false-positive likelihood**: The modeled probability of the evidence if the location is not at risk.

*fixture* – **offline road-graph fixture**: A versioned stored graph snapshot used for reproducible Demonstration Mode without a live map request.

$G = (V, E)$  – **city graph**: The finite undirected graph whose vertices represent locations or movement connectors and whose edges represent roads.

$p$  – **daily transmission probability**: A bounded scenario probability used in binomial infection trials.

$I$  – **infected population**: Modeled currently infectious people at a location.

$P(E|H)$  – **likelihood**: The modeled probability of observed symptom evidence if the location is at risk.

$(S, T)$  – **minimum cut**: A partition of a flow network whose crossing edges have minimum total capacity.

*OSM* – **OpenStreetMap**: The volunteered geographic-data source used only for bounded road geometry.

*Overpass* – **Overpass API**: A public query service used as the primary bounded OSM road-data route.

$P(H|E)$  – **posterior probability**: The normalized Bayesian risk score after applying local evidence.

$P(H)$  – **prior probability**: The location-risk probability before current symptom evidence is incorporated.

**projection – scenario-model projection:** An output conditional on the stated synthetic inputs, not a real-world forecast.

**$R$  – recovered population:** People removed from the infectious state through the model’s recovery transition.

**$\sigma$  – simulation seed:** An integer controlling deterministic pseudo-random event draws.

**$d(s, t)$  – shortest path:** The least total transformed edge cost between source and destination.

**$SIR$  – susceptible–infected–removed:** The compartment vocabulary used for count updates: susceptible, infected, recovered, and deceased.

**$S$  – susceptible population:** People who have not yet been infected in the stated scenario.

**$tie$  – tie-aware winner status:** A comparison result with equal final infections and modeled deaths for more than one strategy.

**$TTL$  – time to live:** The in-memory cache duration for a normalized live road import.

**$v$  – vertex:** A graph location; in EpiGraph it may be a population-bearing facility or a zero-population connector.